PSYLOCKE: Provably Secure Logic Locking with Practical Efficiency

Yohei Watanabe[1,2], Kyoichi Asano[1,2], Haruka Hirata[1], Tomoki Ono[1], Mingyu Yang[3], Mitsugu Iwamoto[1], Yang Li[1], and Yuko Hara[3]

[1] The University of Electro-Communications, Tokyo, Japan.
{watanabe, k.asano, h.haruka, onotom, mitsugu, liyang}@uec.ac.jp
[2] National Institute of Advanced Industrial Science and Technology, Tokyo, Japan.
[3] Institute of Science Tokyo, Tokyo, Japan.
{mingyu, hara}@cad.ict.e.titech.ac.jp

June 13, 2025

Abstract

Logic locking is an obfuscation technique designed to protect the intellectual property of hardware designs and has attracted considerable attention for over a decade. However, most logic locking schemes have been developed heuristically, leading the field into a cat-and-mouse game between attackers and defenders. Indeed, several proposed schemes have already been broken. While recent works have introduced *provably secure* logic locking, they often incur impractical overhead or fail to support the ASIC design paradigm while offering strong theoretical security guarantees.

In this work, we propose PSYLOCKE, a provably secure and practically efficient logic locking scheme that balances formal security guarantees with implementation feasibility. We introduce a new security paradigm that formalizes logic locking under predetermined allowable leakage, such as circuit topology, and we provide refined definitions of resilience against specific attacks. Our analysis bridges general security definitions and attack resilience, quantifying how leakage impacts the success of real-world attacks. PSYLOCKE is provably secure under topology leakage and achieves significant efficiency improvement compared to existing provably secure logic locking schemes. Alongside our theoretical analysis, we demonstrate through quantitative evaluations using widely-used benchmark circuits that PSYLOCKE strikes a favorable balance between practical performance and security. Concretely, PSYLOCKE reduced the Area-Power-Delay overhead by an order of magnitude while achieving the same security level, compared to the existing provably secure logic locking scheme.

Contents

1 Introduction

With advances in semiconductor-related technologies, the cost of managing and maintaining integrated circuit (IC) fabrication facilities has become drastically increasing. As a result, many semiconductor companies have adopted a fabless model, outsourcing the manufacturing of IC chips to third-party foundries, assemblers, etc. However, such third-party entities cannot always be fully trusted. Semiconductor companies face a range of threats, including reverse engineering, counterfeiting, theft of circuit design information, which constitutes valuable intellectual property (IP), and overproduction of ICs based on stolen designs [MAS+21]. Among these threats, circuit design information is particularly critical due to its high commercial value, making it essential to protect IP even in the condition of outsourcing chip fabrication.

Logic locking [RKM08] has been proposed to address these challenges, and a wide range of research has since been conducted in this area, e.g., [BTZ10, WSM+16, YSN+17, MZG+18, KKN+19, CZH+21, BGH+22, MJS+22, MGM+22]. Logic locking enables the transformation of a circuit $\mathbb{C}$ into a locked circuit $\mathbb{L}$ using a secret key. As shown in Figure 1, by providing the design of $\mathbb{L}$ to third-party entities, a locked IC chip can be manufactured and assembled. This approach allows the simultaneous achievement of IP protection and secure outsourcing of IC fabrication. Without knowledge of the key, it is infeasible to recover the original circuit $\mathbb{C}$ from $\mathbb{L}$. Meanwhile, the semiconductor company (or designer) can unlock (or activate) the chip using the correct key to restore the functionality of the original circuit $\mathbb{C}$.

Cat-and-mouse Game of Logic Locking. Logic locking designs have been developed experimentally in many previous works, e.g., [RKM08, RPS+12, DBD+14, PM15, RZZ+15, WSM+16, MZG+18, KAH+19, KKN+19, MAS+21, APM+23]. Researchers propose new logic locking schemes and demonstrate experimental resilience to specific attacks, only for other researchers to later develop successful attacks on those schemes. In earlier years, ad-hoc or heuristic insertion of dummy gates (e.g., XOR/XNOR gates [RKM08, RPS+12, RZZ+15], AND/OR gates [DBD+14], and multiplexers [PM15, RZZ+15]) into the original design have been proposed, but they were soon broken by attacks exploiting the algorithmic weakness of these logic locking algorithms (e.g., sensitization attack [RPS+12] and SAT attack [SRM15]). Especially the SAT attack [SRM15] is one of the strongest key recovery attacks to effectively reduce candidate logic locking keys. Consequently, various SAT-resilient logic locking methods (e.g., SFLL [YSN+17] and its variants, cyclic logic locking [CCJ+20]) have also been proposed to thwart the SAT attack by letting only one candidate key be excluded per iteration, rendering the attack effort exponential in the number of bits in the secret key. However, because structural weakness remains in the design locked by these SAT-resilient logic locking methods, they were also broken by removable attacks exploiting the structural characteristics [YTS19, YMS+20, LPS22, SCM+22] or preprocessing to unroll the cyclic loops prior to the SAT attack [SPJ19a]. This cat-and-mouse cycle has been a recurring pattern in the field.

1.1 Provable Security of Logic Locking

To overcome the limitations of heuristic logic locking designs, efforts have also been made to formalize the security of logic locking and to provide theoretical guarantees [YSN+17, ZSL+18, SPJ19b, BGH+22, CS22, MJS+22, MGM+22]. These research efforts can be broadly categorized into two types: (1) *attack resilience*: those that provide theoretical analysis against specific types of attacks [YSN+17, ZSL+18, SPJ19b, CS22, MJS+22], and (2) *general security definition*: those that aim to define security from a cryptographic perspective, seeking constructions that are secure against general classes of attacks [BGH+22, MJS+22, MGM+22]. Among the latter, the

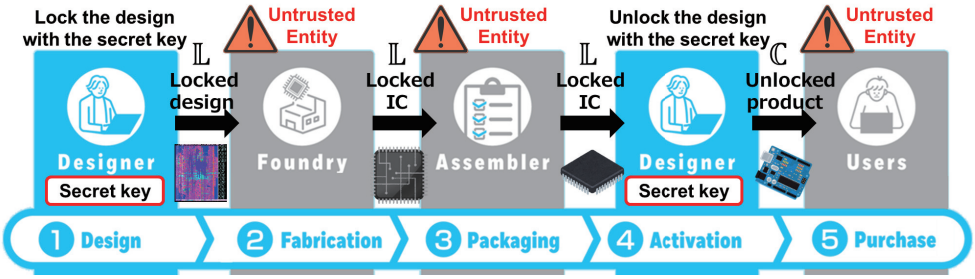


Figure 1: A simplified IC supply chain.

formalisms proposed by Beerel et al. [BGH+22] and Masserova et al. [MGM+22] stand out as successfully capturing the essential characteristics and threat models of logic locking in cryptographic terms. They provided definitions that ensure no information about the original circuit $\mathbb{C}$ is leaked under any attack.

The Limitations of Existing Works. Although provable security is the most promising way to avoid the cat-and-mouse game, unfortunately, both of the aforementioned approaches still have significant limitations.

Several works [YSN+17, ZSL+18, SPJ19b, CS22, MJS+22] have attempted to formalize attack resilience against specific types of attacks, such as SAT attacks and structural attacks; however, most of those definitions are unfortunately ill-defined [BGH+22]. Furthermore, from a security perspective, this line of research alone is insufficient. In fact, various structural attacks (e.g., [AFB19, YMS+20, SZ22]) have been proposed thus far. In particular, Limaye et al. [LPS22] showed that structural attacks can still be effective against logic locking schemes that are SAT-attack resilient. This highlights the need for broader and more comprehensive security analyses that go beyond resilience to a specific class of attacks.

The general security definitions proposed by this line of the previous works [BGH+22, MJS+22, MGM+22] are extremely strong, offering the highest level of protection. To achieve such an ideal level of security, Beerel et al. [BGH+22] and Masserova et al. [MGM+22] independently proposed constructions that meet their definitions using universal circuits (UC) [Val76, LYZ+21, DGS+23], which is a "generic" circuit that, given a suitable input, can emulate any target circuit. In their approach, the locked circuit $\mathbb{L}$ is implemented directly as a UC. El Massad et al. [MJS+22] gave a more "cryptography-oriented" construction using an indistinguishability obfuscation ($i\mathcal{O}$) [BGI+12] and pseudorandom function [GGM86]. In this paper, we call the above constructions UCLock and IndLock, respectively, for convenience.

However, those constructions are either extremely inefficient or lack the original motivation of logic locking that the designers want to protect "ASIC" designs. IndLock [MJS+22] incurs significant circuit size overhead due to the use of $i\mathcal{O}$; for example, the authors demonstrated that the resulting locked circuits are four orders of magnitude larger than the original circuit size for well-known benchmark circuits. UCLock [BGH+22, MGM+22] also introduces substantial circuit size overhead, though it is much smaller than that of IndLock. Moreover, implementing UCLock in hardware effectively requires designing the original circuit $\mathbb{C}$ for an FPGA platform. This contradicts the original motivation of logic locking, which was intended to protect ASIC designs during fabrication outsourcing. If the design is implemented on an FPGA, the need for fabrication outsourcing is effectively eliminated.

Challenges. As discussed above, no existing logic locking scheme to date offers both (i) provable security against a sufficiently broad class of attacks and (ii) practical implementability within the

ASIC design paradigm. *Bridging this gap is a key challenge that this paper aims to address.*

One promising approach to overcoming the limitations of existing works is to relax general security definitions [BGH+22, MJS+22, MGM+22] by allowing some predetermined leakage that appears inconsequential to adversaries, in order to preserve practical efficiency. This approach, which seeks to strike a meaningful trade-off between security and efficiency, is already well-established in various areas of cryptography, such as searchable encryption [CGK+06, KPR12]. Indeed, Beerel et al. [BGH+22] adopted this strategy by introducing a general security definition incorporating allowable leakage $\mathcal{L}$.[1] Namely, their definition guarantees that a logic locking scheme leaks no information on a circuit $\mathbb{C}$ beyond the leakage $\mathcal{L}(\mathbb{C})$. However, their analysis was limited to minimal leakage in the context of logic locking, specifically, the number of inputs and outputs of the original circuit. Thus, the definition should be extended to handle a broader range of leakage types, enabling a more comprehensive and realistic assessment of security.

The next step toward achieving our goal is to clarify the relationship between general security definitions and resilience against specific types of attacks. However, even if we succeed in proving that a general security definition, parameterized by some allowable leakage $\mathcal{L}$, implies resilience against a specific attack (e.g., SAT attacks), it remains unclear how impactful the leakage $\mathcal{L}$ is in practice. This is because $\mathcal{L}$ is explicitly permitted to be leaked to adversaries and is therefore excluded from the scope of such implications. In other words, the leakage $\mathcal{L}$ might still be useful to adversaries in mounting SAT attacks or other types of attacks, even if a logic locking scheme satisfies the general security definitions under the leakage $\mathcal{L}$.

Our Goal. To summarize, in this paper, we aim to propose a *new logic locking scheme that is provably secure against a broad class of attacks while also achieving practical efficiency and implementability*. To this end, we pursue two specific goals. First, we establish a new paradigm for formalizing and analyzing the security of logic locking under allowable leakage $\mathcal{L}$, enabling a more realistic and flexible evaluation of security. Second, we construct a new logic locking scheme that adheres to this paradigm, demonstrating both provable security guarantees and practical implementability, particularly within the ASIC design framework.

1.2 Our Contributions

In this paper, we successfully accomplish our goal outlined above. Specifically, we establish a new paradigm to formalize and analyze the security of logic locking under allowable leakage, and we propose PSYLOCKE, a provably secure logic locking scheme that also achieves practical efficiency.

New Paradigm for Security Formalization of Logic Locking. We begin by revisiting existing formalizations of logic locking and propose a new security definition that explicitly accommodates arbitrary forms of allowable leakage $\mathcal{L}$. Our definition builds upon the framework introduced by Beerel et al. [BGH+22], but extends it to provide a more rigorous treatment of leakage-aware security. Specifically, we guarantee that a logic locking scheme leaks no information beyond the predefined leakage function $\mathcal{L}$.[2]

Although our security definition precisely specifies what information is leaked to adversaries via the leakage function $\mathcal{L}$, it remains unclear how useful for the adversaries this leakage is in practice, specifically, whether it can facilitate concrete attacks such as SAT or removal attacks. To address this, we analyze the relationship between the predefined leakage and the effectiveness of specific

[1] A concrete example of $\mathcal{L}$ is the number of inputs and outputs of the original circuit. See Section 3 for a detailed explanation of $\mathcal{L}$.

[2] Although Beerel et al. [BGH+22] also incorporated a predefined leakage function $\mathcal{L}$ in their security definition, they did not explore its implications in depth, leaving the analysis of leakage-aware security largely unaddressed.

attacks. In particular, we also revisit two categories of resilience against specific attacks: *key recovery attacks*, which include SAT attacks aimed at recovering the secret key, and *circuit recovery attacks*, which encompass structural attacks such as removal attacks that attempt to reconstruct the original circuit. We then demonstrate the extent to which a logic locking scheme that permits only specific, predefined leakage can withstand these types of attacks. Specifically, we show that:

- If a logic locking scheme minimizes the allowable leakage, i.e., it only consists of the number of inputs and outputs of $\mathbb{C}$, denoted by $\mathcal{L}_{\mathtt{io}}(\mathbb{C})$, its resilience against the key recovery and circuit recovery attacks exponentially depends on the number of inputs.
- If a logic locking scheme only leaks the structural topology of the original circuit $\mathbb{C}$, denoted by $\mathcal{L}_{\mathtt{topo}}(\mathbb{C})$, its resilience against the key recovery and circuit recovery attacks linearly depends on the number of gates in a certain sub-circuit decomposed from $\mathbb{C}$.

We emphasize that our approach of bridging a general security definition with resilience against specific attacks is novel. While El Massad et al. [MJS+22] introduced a security notion for general attacks, called *locked circuit indistinguishability*, and demonstrated that it directly implies resilience to certain specific attacks, our key insight is that the allowable leakage $\mathcal{L}$ provides a crucial link between our security definitions for general classes of attacks and against key recovery and circuit recovery attacks. Namely, we formalize (1) what information a logic locking scheme explicitly leaks, and then (2) how useful that leakage is in enabling specific attacks. We do not consider a direct connection between the logic locking schemes themselves and resilience against specific attacks, since such attacks are more appropriately characterized by the information available to adversaries, rather than by the algorithms of the logic locking schemes.

New Provably Secure Logic Locking Scheme. We propose PSYLOCKE, a provably secure logic locking scheme that incurs significantly lower circuit size overhead compared to existing provably secure approaches such as UCLock and IndLock, while leaking only the circuit topology $\mathcal{L}_{\mathtt{topo}}$. Given that we have shown $\mathcal{L}_{\mathtt{topo}}$ still provides a meaningful level of resilience against both key recovery and circuit recovery attacks, PSYLOCKE inherits this resilience under the leakage. PSYLOCKE utilizes universal gates [KS16, LMS16, LYZ+21], a logic-gate version of UC, as its building block to ensure that no other information about $\mathbb{C}$ is leaked beyond its topology $\mathcal{L}_{\mathtt{topo}}(\mathbb{C})$. PSYLOCKE significantly improves efficiency in terms of the number of gates compared to existing provably secure logic locking schemes, IndLock [MJS+22] and UCLock [BGH+22, MGM+22]. Specifically, for well-known benchmark circuits, PSYLOCKE achieves a 10,000x smaller overhead compared to IndLock, and a 3.5x to 4x smaller overhead compared to UCLock.

Security and Performance Evaluation. We evaluated various logic locking methods in terms of their resilience to SAT and structural attacks and the circuit area-power-delay (APD) overhead. All methods, including AntiSAT, CAC, and SARLock, resisted SAT attacks with a 128-bit key under a 48-hour time-out. UCLock and PSYLOCKE also demonstrated SAT resilience. However, structural attacks using Valkyrie [LPS22] successfully broke AntiSAT, CAC, and SARLock due to the presence of structural weakness. In contrast, PSYLOCKE and UCLock resisted Valkyrie's attacks since their structure-neutral approach to lock the original circuit could inherently defeat structural attacks. Moreover, PSYLOCKE reduced the APD overhead by an order of magnitude compared to UCLock, demonstrating its implementation efficiency.

1.3 Notations

As in previous works on provably secure logic locking [YSN+17, BGH+22, MGM+22], we focus on combinational circuits, which consist solely of logic gates and interconnecting wires and do not

contain any feedback loops. Combinational circuits can be represented as directed acyclic graphs (DAGs), where each node corresponds to a logic gate. We define the circuit size $\ell := \ell_i + \ell_o + \ell_g$ for ℓ_i-input ℓ_o-output circuits composed of ℓ_g gates.

For a finite set $\mathcal{X}$, we use $x \xleftarrow{\$} \mathcal{X}$ to represent processes of choosing an element x from X uniformly at random. For a finite set $\mathcal{X}$, we denote by $\mathcal{X} \leftarrow x$ and $|\mathcal{X}|$ the addition x to $\mathcal{X}$ and cardinality of $\mathcal{X}$, respectively. For any algorithm A, $\mathsf{out} \leftarrow \mathsf{A}(\mathsf{in})$ means that A takes in as input and outputs out. We denote by A^{O} A allowed access to an oracle O. Throughout the paper, we denote by λ a security parameter. We say a function $\mathsf{negl}(\cdot)$ is negligible if for any polynomial $\mathsf{poly}(\cdot)$, there exists some constant $\lambda_0 \in \mathbb{N}$ such that $\mathsf{negl}(\lambda) < 1/\mathsf{poly}(\lambda)$ for all $\lambda \geq \lambda_0$.

1.4 Outline

The remainder of this paper is organized as follows. Section 2 briefly reviews existing logic locking schemes. Sections 3 and 4 define our general security and revisited attack resilience, respectively. Section 5 presents the construction of our provably secure logic locking scheme, PSYLOCKE. Section 6 quantitatively evaluates the security and implementation efficiency of PSYLOCKE for widely-used benchmark circuits compared to existing logic locking schemes. Finally, Section 7 concludes this paper.

2 Logic Locking

2.1 Syntax

Following previous works [BGH+22, MGM+22], we define the syntax of logic locking schemes and the correctness below.

Definition 1 (Logic Locking)**.** *A logic locking scheme consists of a probabilistic algorithm* Lock *defined below.*

- $(\mathbb{L}, \mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C})$*: It takes a security parameter* λ *and* ℓ_i*-input* ℓ_o*-output circuit* $\mathbb{C} : \{0,1\}^{\ell_i} \to \{0,1\}^{\ell_o}$ *as input, and outputs a* locked *circuit* $\mathbb{L} : \{0,1\}^{\ell_i} \times \{0,1\}^{\kappa} \to \{0,1\}^{\ell_o}$ *and a key* $\mathsf{k} \in \{0,1\}^{\kappa}$*.*

Definition 2 (Correctness)**.** *For all* $\lambda \in \mathbb{N}$*, all* $\ell_i, \ell_o \in \mathbb{N}$*, all* ℓ_i*-input* ℓ_o*-output circuits* $\mathbb{C}$*, all* $(\mathbb{L}, \mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C})$*, all* $x \in \{0,1\}^{\ell_i}$*, it holds* $\mathbb{L}(x, \mathsf{k}) = \mathbb{C}(x)$*.*

2.2 Threat Model

We consider third-party foundries, which manufacture IC chips outsourced by semiconductor companies, as potential adversaries. The adversary receives the netlist of a locked circuit $\mathbb{L}$ and fabricates it. Once the chip is activated by the semiconductor company using the secret key k, i.e., $\mathbb{L}(\cdot, \mathsf{k})$, it is distributed to the market. The adversary can then obtain a chip and observe its input-output behavior through black-box (oracle) access.

The adversary's goal is to recover the protected IP, such as the circuit's design or functionality, from the locked circuit. Specifically, as in prior work on logic locking, the adversary attempts to construct a counterfeit circuit $\mathbb{C}^*$ that has the same functionality as the original one $\mathbb{C}$, i.e., $\mathbb{C}^*(x) = \mathbb{C}(x)$ holds for all inputs x, using both the information from the locked netlist $\mathbb{L}$ (later formalized as *allowable leakage*) and oracle access to the activated chip.

2.3 New Paradigm for Security Formalization

As described in the introduction, there are two types of provable security of logic locking:

- *Attack resilience* [YSN+17, ZSL+18, CSS+19, SPJ19b, CS22, MJS+22]: It focuses on analyzing the robustness of a scheme against a specific type of attack, such as SAT-based attacks. Although various practically efficient constructions have been proposed that satisfy this notion, the definition does not offer security guarantees against other types of attacks, limiting its applicability in broader adversarial models. Indeed, most provably secure logic locking schemes proposed in those works have been broken by structural attacks [LPS22].

- *General security definition* [BGH+22, MJS+22, MGM+22]: It aims to formalize security against a broad class of attacks, rather than targeting specific attack vectors. Prior works in this area have primarily focused on very strong security settings, where no information about the circuit $\mathbb{C}$ is leaked beyond minimal leakage, such as its number of inputs and outputs. While this approach offers strong theoretical guarantees and naturally encompasses all specific types of attacks, the resulting constructions are typically highly inefficient and impractical for real-world deployment.

In this work, we aim to formalize and quantify the security of logic locking *even in the presence of a certain allowable leakage*, extending beyond the minimal leakage. Our goal is *to enable the design of new logic locking schemes that simultaneously achieve provable security and practical efficiency*.

To this end, we introduce a new paradigm for the security formalization of logic locking as follows.

(i) We define general security definition under predetermined allowable leakage $\mathcal{L}$, which guarantees that the procedure and structure of a logic locking scheme Lock and the resulting locked circuit $\mathbb{L}$ leak no information beyond the leakage $\mathcal{L}$ against *any adversary*. The detailed definition will be given in Section 3.

(ii) We revisit and define resilience against specific attacks under given allowable leakage $\mathcal{L}$. In particular, we focus on two important attack categories: *key recovery attacks* and *circuit recovery attacks*, which can be viewed, roughly speaking, as generalizations of SAT attacks and structural attacks, respectively. Notably, the definitions of these resilience notions are entirely characterized by the leakage function $\mathcal{L}$, and are independent of the specific construction of the Lock algorithm. This is because our general security definition already guarantees what information Lock is allowed to leak, enabling a clean separation between leakage characterization and attack resilience analysis. The detailed definitions and analysis will appear in Section 4.

3 Our General Security Definition

Previous works [BGH+22, MJS+22, MGM+22] have proposed general security definitions for logic locking, but in different formulations. Masserova et al. [MGM+22] introduced a simulation-based security definition grounded in computational indistinguishability, while El Massad et al. [MJS+22] put forward an indistinguishability-based definition. A common limitation of both formalizations is that they did not explicitly consider leakage; instead, they implicitly assumed minimal leakage, typically restricted to the number of inputs and outputs of the circuit $\mathbb{C}$.

Our general security definition closely follows that of Beerel et al. [BGH+22], whose work was among the first to explicitly incorporate the notion of "leakage" into the formalization of logic locking security. However, their analysis was confined to minimal leakage, i.e., the number of inputs and outputs, and did not investigate broader forms of allowable leakage that may arise in more practical or relaxed settings.

Allowable Leakage $\mathcal{L}$. As demonstrated in previous works on *ideally secure* logic locking schemes such as UCLock [BGH+22, MGM+22] and IndLock [MJS+22], it appears inherently challenging to achieve both practical efficiency and strong security under the constraint of minimal leakage. To address this, following the approach of Beerel et al., we adopt the notion of a leakage function $\mathcal{L}$, which takes a circuit $\mathbb{C}$ as input and outputs specific information about $\mathbb{C}$ that is permitted to be leaked to adversaries.

In this paper, we primarily focus on a specific instance of such leakage: the topology leakage $\mathcal{L}_{\mathtt{topo}}$, which reveals the structural topology of the circuit $\mathbb{C}$. In addition, we also take a look at the minimal leakage $\mathcal{L}_{\mathtt{io}}$, which only discloses the number of inputs and outputs, for our theoretical analysis.

Our Definition. We introduce a slight modification to Beerel et al.'s definitions by allowing the parameter δ to be a function of the security parameter λ, rather than a fixed constant as in the original formulation. This minor adjustment enables the definition to encompass both computational security and information-theoretic security settings, since δ can now represent either a negligible function (for computational security) or a constant (for information-theoretic security).

Definition 3 ($(t, \delta, \mathcal{L})$-IND-LL Security)**.** *A logic locking scheme* Lock *is said to meet* $(t, \delta, \mathcal{L})$-IND-LL *security if for some* $\lambda_0 \in \mathbb{N}$*, any* $\lambda \geq \lambda_0$*, and any* t*-time adversary* A *and any circuits* $\mathbb{C}_0$ *and* $\mathbb{C}_1$*, it holds that*

$$\left| \Pr \left[\begin{array}{c} b \xleftarrow{\$} \{0,1\} \\ (\mathbb{L}, \mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C}_b) \\ b' \leftarrow \mathsf{A}(\mathsf{st}, \mathbb{L}, \mathcal{L}(\mathbb{C}_b)) \end{array} : b' = b \right] - \frac{1}{2} \right| \leq \delta(\lambda),$$

where $\mathbb{C}_0$ *and* $\mathbb{C}_1$ *satisfies* $\mathcal{L}(\mathbb{C}_0) = \mathcal{L}(\mathbb{C}_1)$*.*

As Beerel et al. mentioned, no oracle (i.e., a working chip $\mathbb{C}$) appears in the above IND-LL security game since A already knows the two candidates, $\mathbb{C}_0$ and $\mathbb{C}_1$, of the original circuits. In contrast, the following simulation-based security allows adversaries to access the oracle.

Definition 4 ($(t, \delta, \mathcal{L})$-SIM-LL Security)**.** *A logic locking scheme* Lock *is said to meet* $(t, \delta, \mathcal{L})$-SIM-LL *security if for some* $\lambda_0 \in \mathbb{N}$*, any* $\lambda \geq \lambda_0$*, and any* t*-time adversary* A*, there exists a* $\mathsf{poly}(t)$*-time simulator* S *such that for any circuit* $\mathbb{C}$*, it holds that*

$$\left| \Pr \left[(\mathbb{L}, \mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C}) : 1 \leftarrow \mathsf{A}^{\mathbb{C}}(\mathbb{L}, \mathcal{L}(\mathbb{C})) \right] - \Pr \left[1 \leftarrow \mathsf{S}^{\mathbb{C}}(1^\lambda, \mathcal{L}(\mathbb{C})) \right] \right| \leq \delta(\lambda).$$

As Beerel et al. showed, it holds that $(t, \delta, \mathcal{L})$-IND-LL security implies $(t, \delta, \mathcal{L})$-SIM-LL security, despite some minor differences between their formulation and ours. The proofs closely follow those provided by Beerel et al. [BGH+22], and are therefore omitted for brevity.

Proposition 1 ($(t, \delta, \mathcal{L})$-IND-LL $\Rightarrow$ $(t, \delta, \mathcal{L})$-SIM-LL [BGH+22])**.** *Let* Lock *be a logic locking scheme satisfying correctness. If* Lock *is* $(t, \delta, \mathcal{L})$-IND-LL *secure,* Lock *is also* $(t, \delta, \mathcal{L})$-SIM-LL *secure.*

Proposition 2 ($(t, \delta, \mathcal{L})$-SIM-LL $\not\Rightarrow$ $(t, \delta, \mathcal{L})$-IND-LL [BGH+22])**.** *Let* Lock *be a logic locking scheme satisfying correctness. If* Lock *meets* $(t, \delta, \mathcal{L})$-SIM-LL *security,* Lock *does not always satisfy* $(t, \delta, \mathcal{L})$-IND-LL *security.*

4 Definitions of Attack Resilience, Revisited

In general, an adversary against logic locking schemes aims to ideally recover a circuit $\mathbb{C}$ from a netlist, i.e., a locked circuit $\mathbb{L}$ and other available information. In the literature, specific approaches to doing so are classified into two major attacks [MJS+22]: *key recovery attacks* and *circuit recovery attacks*.

4.1 Resilience against Key Recovery Attacks

Key recovery attacks aim to recover *valid* secret keys. SAT attacks [MGT15, SRM15] are known as the most effective key recovery attacks against logic locking schemes. An adversary A has a netlist of a chip $\mathbb{C}$, i.e., a locked circuit $\mathbb{L}$ of $\mathbb{C}$, and is allowed to access the working chip $\mathbb{C}$, which is modeled as an oracle that returns outputs of the chip for queried input patterns. The ultimate goal of the adversary A who knows $\mathcal{L}(\mathbb{C})$ is to find k^* such that

$$\forall x \in \{0,1\}^{\ell_i},\ \mathbb{L}(x, \mathsf{k}^*) = \mathbb{C}(x).$$

Previous Definition and Its Limitations. Several attempts have been made to formalize resilience against key recovery attacks [YSN+17, ZSL+18, CS22, MJS+22], particularly SAT attacks; however, most of these formalisms [YSN+17, ZSL+18, CS22] are ill-defined, as noted by Beerel et al. [BGH+22]. Yasin et al.'s definition of SAT-attack resilience [YSN+17] aimed to formalize security against so-called "SAT attack adversaries"; however, these adversaries were not formally defined, and it appears inherently difficult to do so in a rigorous manner. Zaman et al. [ZSL+18] adopted the security definition proposed by Yasin et al., and as a result, they fell into the same pitfall. Chhotaray and Shrimpton [CS22] introduced a security notion against key recovery attacks for a function class $\mathcal{F}$. However, as Beerel et al. pointed out, their definition imposes a strong requirement: the adversary must output a circuit that is functionally identical to the original, which may be unrealistic or unnecessarily restrictive in practical settings.

The only exception is a definition by El Massad et al. [MJS+22]. To describe their definition, let us additionally define a variant of the Lock algorithm, since El Massad et al. employed it:

- $\mathbb{L} \leftarrow \mathsf{Lock}'(\mathsf{k}, \mathbb{C})$: It takes a uniformly-chosen secret key $\mathsf{k} \xleftarrow{\$} \{0,1\}^\lambda$ and ℓ_i-input ℓ_o-output circuit $\mathbb{C}$ as input, and outputs a locked circuit $\mathbb{L}$.

Definition 5 (Security against Key Recovery Attacks [MJS+22])**.** *A logic locking scheme* Lock *is secure against key recovery attacks if there exists a negligible function* negl *such that for any polynomial-time adversary* A*, a polynomial-time simulator* S *exists such that for every circuit* $\mathbb{C}$*, every sufficiently large* $\lambda \in \mathbb{N}$*, and every polynomial-time algorithm* V *that outputs* 1 *only if it holds*

$$\left| \Pr\left[\begin{array}{l} \mathsf{k} \xleftarrow{\$} \{0,1\}^\lambda \\ \mathbb{L} \leftarrow \mathsf{Lock}'(\mathsf{k}, \mathbb{C}) : \mathsf{V}(\mathbb{L}(\mathsf{k}_\mathsf{A}, \cdot), \mathbb{C}) \to 1 \\ \mathsf{k}_\mathsf{A} \leftarrow \mathsf{A}^{\mathbb{C}}(\mathbb{L}) \end{array}\right] - \Pr\left[\begin{array}{l} \mathsf{k} \xleftarrow{\$} \{0,1\}^\lambda \\ \mathbb{L} \leftarrow \mathsf{Lock}'(\mathsf{k}, \mathbb{C}) \\ \mathsf{k}_\mathsf{S} \leftarrow \mathsf{S}^{\mathbb{C}}(\mathcal{L}_{io}(\mathbb{C}), 1^\lambda) \end{array} : \mathsf{V}(\mathbb{L}(\mathsf{k}_\mathsf{S}, \cdot), \mathbb{C}) \to 1\right] \right| \leq \mathsf{negl}(\lambda).$$

Observation. Intuitively, the above definition ensures that any V cannot efficiently find an input x^* such that $\mathbb{L}(\mathsf{k}_\mathsf{A}, x^*) \neq \mathbb{L}(\mathsf{k}_\mathsf{S}, x^*)$ within polynomial time in λ. Consequently, the locked circuit $\mathbb{L}$ leaks only the number of inputs and outputs $\mathcal{L}_{\mathtt{io}}(\mathbb{C})$, but no additional information about the functionality of the original circuit $\mathbb{C}$.

Experiment: $\mathsf{Exp}^{\gamma\text{-}\mathsf{KRA}}_{\mathsf{A},\mathcal{L}}(1^\lambda, \mathbb{C})$

1: $(\mathbb{L}, \mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C})$ // $\mathcal{K}_\mathbb{C}$ is fixed
2: $\mathcal{Y} := \emptyset$
3: $x \leftarrow \mathsf{A}(\mathbb{L}, \mathcal{L}(\mathbb{C}), \mathcal{Y})$ // choose a query to an oracle
4: $\mathcal{Y} \leftarrow (x, \mathbb{C}(x))$
5: **for** $\forall \widetilde{\mathsf{k}} \in \mathcal{K}_\mathbb{C}(\mathcal{Y})$ **do**
6: **if** $\max\left\{ \frac{|\mathcal{X}|}{2^{\ell_i}} \;\middle|\; \mathcal{X} \text{ s.t. } \mathbb{L}(x, \widetilde{\mathsf{k}}) = \mathbb{C}(x) \text{ for } \forall x \in \mathcal{X} \right\} \leq \gamma$ **then**
7: Go back to line 3 // since there still exists a wrong key
8: **return** $|\mathcal{Y}|$

Figure 2: The KRA resilience game for an ℓ_i-input ℓ_o-output circuit $\mathbb{C}$. $\mathcal{K}_\mathbb{C}$, which is determined by running $\mathsf{Lock}(1^\lambda, \mathbb{C})$ and the predetermined leakage $\mathcal{L}(\mathbb{C})$, represent the sets of possible keys for $\mathbb{C}$. For any set $\mathcal{Y} := \{(x_i, \mathbb{C}(x_i))\}_{i=1}^{|\mathcal{Y}|}$, let $\mathcal{K}_\mathbb{C}(\mathcal{Y}) \subset \mathcal{K}_\mathbb{C}$ be a set of possible keys induced by $\mathcal{K}_\mathbb{C}$, $\mathcal{Y}$, and $\mathcal{L}(\mathbb{C})$.

While their definition may appear practically plausible, it exhibits several notable gaps, particularly in quantifying the extent to which the allowable leakage contributes to the success of key recovery attacks, as outlined below.

- In practice, secret keys in almost all logic locking schemes are *not* uniformly distributed. Indeed, secret keys are not uniformly chosen in several both heuristically and provably secure schemes (e.g., [DBD+14, MZG+18, KKN+19, BGH+22, MGM+22]). Therefore, their definition relies on the unreasonable assumption of key uniformity.

- Computational indistinguishability typically relies on computational complexity assumptions, such as the integer factoring assumption. While such assumptions are widely used and useful for constructing a variety of cryptographic protocols, they can lead to significant practical inefficiencies in the context of logic locking, where low overhead and implementation efficiency are critical. For example, IndLock achieves strong security guarantees by leveraging $i\mathcal{O}$, however, circuit sizes increase by approximately 10,000× for widely used benchmark circuits (see also Table 1), making such schemes impractical for real-world deployment.

- The definition only captures the key recovery attack under the minimum leakage $\mathcal{L}_{\mathtt{io}}$. While it can be generalized by substituting $\mathcal{L}_{\mathtt{io}}$ with an arbitrary leakage function $\mathcal{L}$, the definition merely implies that the locked circuit $\mathbb{L}$ is no more helpful for key recovery than the information explicitly revealed by $\mathcal{L}$. However, it still does not quantify or clarify how useful the allowable leakage $\mathcal{L}$ is in actually enabling a key recovery attack, leaving a gap in understanding the practical implications of the leakage.

Our Definition. How, then, can we meaningfully quantify the level of resilience against key recovery attacks, given the above discussion? It is important to quantify the impact of the allowable leakage $\mathcal{L}$ for key recovery attacks, since our general security definition (Definition 4) can ensure a logic locking scheme leaks no information other than $\mathcal{L}$.

Let us consider an experiment $\mathsf{Exp}^{\gamma\text{-}\mathsf{KRA}}_{\mathsf{A},\mathcal{L}}$ involving an adversary A attempting to perform key recovery attacks, as given in Figure 2. In general, the goal of A is to identify a set of *valid keys*[3]

[3]A secret key k^* is said to be *valid* if $\mathbb{L}(x, \mathsf{k}^*) = \mathbb{C}(x)$ holds for all $x \in \{0,1\}^{\ell_i}$. Note that multiple valid keys may exist, not just the one originally output by the Lock algorithm.

from the key space $\mathcal{K}_{\mathbb{C}}$ for a given circuit $\mathbb{C}$, by issuing queries to an oracle $\mathbb{C}$ and analyzing the allowable leakage $\mathcal{L}(\mathbb{C})$. Therefore, *a critical aspect of this experiment is to determine the number of input-output pairs required for the adversary to succeed in this task*, since this metric directly reflects the effectiveness of the allowable leakage in facilitating key recovery.

Moreover, we consider *approximate attacks* [SLM+17] by relaxing the adversary's success condition in $\mathsf{Exp}^{\gamma\text{-}\mathsf{KRA}}_{\mathsf{A},\mathcal{L}}$. Specifically, A is said to succeed in an (approximate) key recovery attack if A can identify a secret key $\widetilde{\mathsf{k}}$ such that the locked circuit $\mathbb{L}(x, \widetilde{\mathsf{k}})$ matches the output of the original circuit $\mathbb{C}(x)$ for at least a fraction γ of the input space $\{0,1\}^{\ell_i}$. This relaxation reflects practical scenarios where partial correctness may still pose a significant security risk.

Formally, our definition for resilience against key recovery attacks is given as follows.

Definition 6 ((t, γ, η)-KRA Resilience under $\mathcal{L}$)**.** *Let* Lock *be a logic locking scheme that satisfies the correctness and only allows the leakage* $\mathcal{L}$*.* Lock *is said to meet* (t, γ, η)*-key-recovery-attack (KRA) resilience under the leakage* $\mathcal{L}$ *if for any* $\lambda \in \mathbb{N}$*, any* t*-time adversary* A*, and for any circuit* $\mathbb{C}$*, it holds that*

$$\mathsf{Exp}^{\gamma\text{-}\mathsf{KRA}}_{\mathsf{A},\mathcal{L}}(1^\lambda, \mathbb{C}) \geq \eta(1^\lambda, \mathbb{C}).$$

Remark 1 (On the parameter η)**.** *We model the number of required queries* η *as a function of the security parameter* λ *and the circuit* $\mathbb{C}$*, as it should naturally depend on factors such as the key size (or the complexity of procedures in the* Lock *algorithm) and the number of inputs and outputs of the circuit.*

4.2 Resilience against Circuit Recovery Attacks

Circuit recovery attacks can be regarded as a generalization of key recovery attacks. Structural attacks are a popular and powerful approach for circuit recovery attacks and include several specific ones, such as removal attacks, depending on their attack strategies. The common aim of the circuit recovery attacks is to find a circuit $\mathbb{C}^*$ such that

$$\forall x \in \{0,1\}^{\ell_i}, \mathbb{C}^*(x) = \mathbb{C}(x),$$

analyzing the functionality or structure of the locked circuit $\mathbb{L}$ with the leakage $\mathcal{L}(\mathbb{C})$.

Previous Definition and Its Limitations. As with key recovery attacks, several attempts have been made to formalize resilience against circuit recovery attacks [YSN+17, CSS+19, SPJ19b, BGH+22, CS22, MJS+22]. However, similar to the case of key recovery, Beerel et al. [BGH+22] pointed out that most of these definitions [CSS+19, SPJ19b, CS22] are ill-defined, highlighting the need for clearer and more precise formulations in the context of resilience against circuit recovery attacks.

We briefly review the remaining existing definitions of resilience against circuit recovery attacks (for detailed discussions, see Appendix A). Yasin et al. [YSN+17] attempted to formalize resilience against removal attacks, a specific type of structural attack. However, their definition involves ambiguous terminology, such as the notion of a "transformation for the removal attack," which lacks precise formal grounding. El Massad et al. [MJS+22] defined a security notion against circuit recovery attacks, defined in a manner similar to their security definition for key recovery attacks (Definition 5). As a result, it inherits the same limitations discussed in the previous section; in particular, it only accounts for the minimal leakage. Beerel et al. [BGH+22] introduced a notion of functional secrecy by refining previous ill-defined security notions [CSS+19, SPJ19b]. Their definition is well-defined in the sense that it avoids trivial attacks that would otherwise render the

Experiment: $\mathsf{Exp}^{\gamma\text{-}\mathsf{CRA}}_{\mathsf{A},\mathcal{L}}(1^\lambda, \mathbb{C})$

1: $(\mathbb{L}, \mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C})$ // $\mathcal{C}_\mathbb{C}$ is fixed
2: $\mathcal{Y} := \emptyset$
3: $x \leftarrow \mathsf{A}(\mathbb{L}, \mathcal{L}(\mathbb{C}), \mathcal{Y})$ // choose a query to an oracle
4: $\mathcal{Y} \leftarrow (x, \mathbb{C}(x))$
5: **for** $\forall \widetilde{\mathbb{C}} \in \mathcal{C}_\mathbb{C}(\mathcal{Y})$ **do**
6: **if** $\max\left\{ \frac{|\mathcal{X}|}{2^{\ell_i}} \;\middle|\; \mathcal{X} \text{ s.t. } \widetilde{\mathbb{C}}(x) = \mathbb{C}(x) \text{ for } \forall x \in \mathcal{X} \right\} \leq \gamma$ **then**
7: Go back to line 3 // since there still exists a wrong circuit
8: **return** $|\mathcal{Y}|$

Figure 3: The CRA resilience game for an ℓ_i-input ℓ_o-output circuit $\mathbb{C}$. $\mathcal{C}_\mathbb{C}$, which is determined by the original circuit $\mathbb{C}$, represent the sets of possible circuits that have the same leakage $\mathcal{L}(\mathbb{C})$, i.e., it holds $\mathcal{L}(\widetilde{\mathbb{C}}) = \mathcal{L}(\mathbb{C})$ for any $\widetilde{\mathbb{C}} \in \mathcal{C}_\mathbb{C}$. For any set $\mathcal{Y} := \{(x_i, \mathbb{C}(x_i))\}_{i=1}^{|\mathcal{Y}|}$, let $\mathcal{C}_\mathbb{C}(\mathcal{Y}) \subset \mathcal{C}_\mathbb{C}$ be a set of possible circuits induced by $\mathcal{C}_\mathbb{C}$, $\mathcal{Y}$, and $\mathcal{L}(\mathbb{C})$.

definition unsatisfiable. However, the functional secrecy notion also excludes the consideration of allowable leakage $\mathcal{L}$, and consequently fails to quantify or clarify the extent to which such leakage may aid in circuit recovery attacks.

Our Definition. We introduce a new definition of resilience against circuit recovery attacks (CRA resilience), formulated analogously to our definition of KRA resilience (Definition 6). Our definition quantifies the impact of the allowable leakage $\mathcal{L}$ by measuring the minimum number of queries to the oracle (the working chip) $\mathbb{C}$ required to perform a successful circuit recovery attack. Moreover, it also accounts for approximate attacks, in the same manner as KRA resilience, thereby capturing both exact and partial recovery scenarios.

Definition 7 ((t, γ, η)-CRA Resilience under $\mathcal{L}$)**.** *Let* Lock *be a logic locking scheme that satisfies the correctness and only allows the leakage* $\mathcal{L}$*.* Lock *is said to meet* (t, γ, η)*-circuit-recovery-attack (CRA) resilience under the leakage* $\mathcal{L}$ *if for any* $\lambda \in \mathbb{N}$*, any* t*-time adversary* A*, and for any circuit* $\mathbb{C}$*, it holds that*

$$\mathsf{Exp}^{\gamma\text{-}\mathsf{CRA}}_{\mathsf{A},\mathcal{L}}(1^\lambda, \mathbb{C}) \geq \eta(1^\lambda, \mathbb{C}),$$

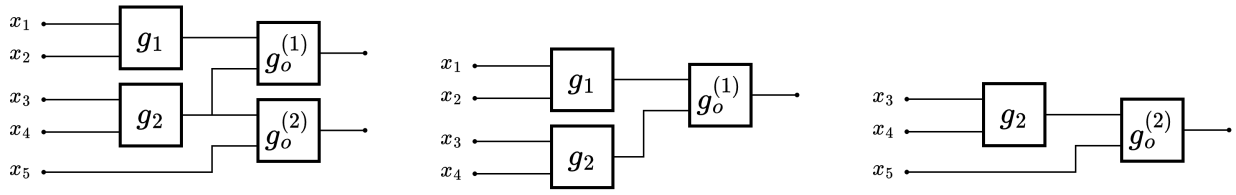
where $\mathsf{Exp}^{\gamma\text{-}\mathsf{CRA}}_{\mathsf{A},\mathcal{L}}$ *is given in Figure 3.*

4.3 What Leakage Ensures Attack Resilience?

The parameter η for security levels of the KRA resilience (Definition 6) and the CRA resilience (Definition 7) is characterized by the allowable leakage $\mathcal{L}$. This means that we may analyze specific values of η only with $\mathcal{L}$, regardless of concrete constructions (or structures) of Lock. In this section, we concretely analyze the extent to which specific forms of leakage, namely, the input/output-size leakage $\mathcal{L}_{\mathtt{io}}$ and the circuit topology leakage $\mathcal{L}_{\mathtt{topo}}$, contribute to the success of key recovery and circuit recovery attacks.

At first, we show that CRA resilience implies KRA resilience.

Theorem 1. *For any leakage* $\mathcal{L}$*, if* (t, γ, η)*-CRA resilience holds under* $\mathcal{L}$*, then* (t, γ, η)*-KRA resilience also holds under* $\mathcal{L}$*.*



(a) Circuit $\mathbb{C}$ **(b)** Fan-in cone $\widetilde{\mathbb{C}}_1$ **(c)** Fan-in cone $\widetilde{\mathbb{C}}_2$

Figure 4: A simple example of a circuit $\mathbb{C}$ and its fan-in cones of $\mathbb{C}$'s primary outputs.

Proof. We show that any adversary $\mathsf{A}_{\mathsf{KRA}}$ against key recovery attacks is also an adversary $\mathsf{A}_{\mathsf{CRA}}$ against circuit recovery attacks. Arbitrarily fix a leakage $\mathcal{L}$, a t'-time adversary $\mathsf{A}_{\mathsf{KRA}}$, a security parameter $\lambda \in \mathbb{N}$, and a circuit $\mathbb{C}$. Suppose that $\mathsf{A}_{\mathsf{KRA}}$ outputs $\eta' := |\mathcal{Y}|$ in $\mathsf{Exp}^{\gamma'\text{-}\mathsf{KRA}}_{\mathsf{A},\mathcal{L}}(1^\lambda, \mathbb{C})$. By definition, for all $\widetilde{\mathsf{k}} \in \mathcal{K}_{\mathbb{C}}(\mathcal{Y})$, it holds that $\mathbb{L}(x, \widetilde{\mathsf{k}}) = \mathbb{C}(x)$ for all input x in some fraction γ' of the input space. This also means that $\mathsf{A}_{\mathsf{KRA}}$ found a circuit $\widetilde{\mathbb{C}} := \mathbb{L}_{\widetilde{\mathsf{k}}}$, and in that sense, $\mathsf{A}_{\mathsf{KRA}}$ is also a CRA adversary $\mathsf{A}_{\mathsf{CRA}}$. By assumption, (t, γ, η)-CRA resilience holds under $\mathcal{L}$, so it must hold that $t' \leq t$, $\gamma' \leq \gamma$, and $\eta' \leq \eta$. Therefore, (t, γ, η)-CRA resilience under the leakage $\mathcal{L}$ implies (t, γ, η)-KRA resilience under the same $\mathcal{L}$. □

Input/Output-Size Leakage $\mathcal{L}_{\mathtt{io}}$. We begin with the minimum allowable leakage $\mathcal{L}_{\mathtt{io}}$. Note that UCLock [BGH+22, MGM+22] and IndLock [MJS+22] achieves this ideal leakage.

Theorem 2. *Let* Lock *be a logic locking scheme satisfying the correctness, and suppose that it allows the leakage $\mathcal{L}_{io}$. Then,* Lock *is $(\infty, \gamma, 2^{\ell_i} \cdot \gamma)$-CRA-resilient for any $\gamma \in (0, 1]$, where ℓ_i is the number of inputs of the circuit $\mathbb{C}$.*

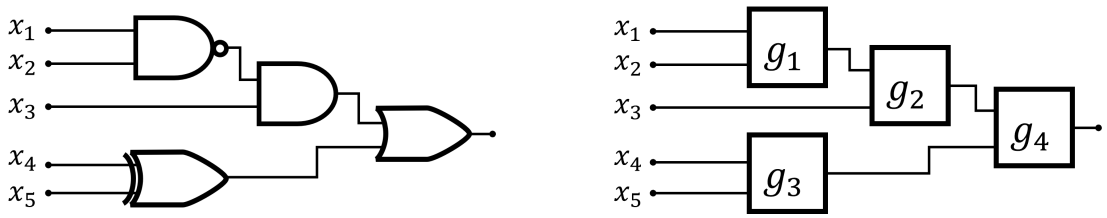
Proof. We arbitrarily fix an adversary A and an ℓ_i-input ℓ_o-output circuit $\mathbb{C}$. Since A is computationally unbounded, it is sufficient to derive the minimum number of queries to identify some fraction γ of the input space $\{0,1\}^{\ell_i}$.

Obviously, no input-output pattern $(x, \mathbb{C}(x))$ provides any information that helps infer other input-output patterns that A has not explicitly queried, since the leakage $\mathcal{L}_{\mathtt{io}}$ reveals only the number of inputs and outputs, and nothing about the internal structure or behavior of circuit $\mathbb{C}$. Thus, the adversary A aiming to recover at least $2^{\ell_i} \cdot \gamma$ input-output patterns has no alternative but to simply issue $2^{\ell_i} \cdot \gamma$ queries, since no additional information is available to aid in inferring unqueried input-output patterns. □

Circuit Topology Leakage $\mathcal{L}_{\mathtt{topo}}$. In the case where circuit topology $\mathcal{L}_{\mathtt{topo}}$ is leaked, the resilience parameter η depends on the number of logic gates in the original circuit $\mathbb{C}$, more precisely, on the number of gates within one of the *fan-in cones* of all outputs of $\mathbb{C}$, which are defined below.

Definition 8 (Fan-In Cones)**.** *Let $\mathbb{C}$ be an ℓ_i-input ℓ_o-output circuit and $g_o^{(1)}, g_o^{(2)}, \ldots, g_o^{(\ell_o)}$ be output gates of $\mathbb{C}$, i.e., gates that outputs each bit of $\mathbb{C}(x)$ for any input $x \in \{0,1\}^{\ell_i}$. For every $j \in \{1, 2, \ldots, \ell_o\}$, the fan-in cone $\widetilde{\mathbb{C}}_j$ of $g_o^{(j)}$ is a sub-circuit of $\mathbb{C}$ such that no part of $\widetilde{\mathbb{C}}_j$ can be removed without compromising its ability to produce the correct output for all possible inputs. For convenience, the fan-in cones $\widetilde{\mathbb{C}}_1, \widetilde{\mathbb{C}}_2, \ldots, \widetilde{\mathbb{C}}_{\ell_o}$ of all outputs of $\mathbb{C}$ are simply called the fan-in cones of $\mathbb{C}$.*

Any ℓ_i-input ℓ_o-output circuit $\mathbb{C}$ can be decomposed into the fan-in cones $\widetilde{\mathbb{C}}_1, \widetilde{\mathbb{C}}_2, \ldots, \widetilde{\mathbb{C}}_{\ell_o}$ of $\mathbb{C}$, where each of them corresponds to each output of $\mathbb{C}$. Please refer to Figure 4, which illustrates



(a) Circuit $\widetilde{\mathbb{C}}$ **(b)** Topology $\mathcal{L}_{\mathsf{topo}}(\widetilde{\mathbb{C}})$

Figure 5: A simple example for the proof of Theorem 3.

a simple example. Therefore, the oracle access allows an adversary to obtain outputs of all fan-in cones of $\mathbb{C}$ with a single input.

Notably, our logic locking construction PSYLOCKE, which will appear in Section 5, only leaks the circuit topology, so it is also CRA-resilient (and hence KRA-resilient) in the following sense.

Theorem 3. *Let* Lock *be a logic locking scheme satisfying the correctness, and suppose that it allows the leakage* $\mathcal{L}_{topo}$*. Then,* Lock *is* $(\mathsf{poly}(\ell), \gamma, \eta)$*-CRA-resilient for any* $\gamma \in (0, 1]$ *and*

$$\eta := \min\{2\widetilde{\ell}_g^* + 2,\ \gamma \cdot 2^{\ell_i}\},$$

where ℓ *is the size of the circuit* $\mathbb{C}$*,* ℓ_i *is the number of inputs of* $\mathbb{C}$*, and* $\widetilde{\ell}_g^*$ *is the maximum number of gates among all fan-in cones of* $\mathbb{C}$*.*

Proof. To enhance readability, we first provide an intuition behind the proof. Suppose that a topology $\mathcal{L}_{\mathsf{topo}}(\widetilde{\mathbb{C}})$ of a simple circuit $\widetilde{\mathbb{C}}$ in Figure 5 and let $\gamma = 1$. We here show how many queries are needed to identify all the input-output pairs for $\widetilde{\mathbb{C}}$. Note that an adversary A only knows the topology $\mathcal{L}(\widetilde{\mathbb{C}})$.

Suppose that A issues the first query $x^{(1)} := (0, 0, 0, 0, 0)$ and obtains $\widetilde{\mathbb{C}}(x^{(1)}) = 0$. A then can create a truth table with variables y_1, y_2, and y_3:

Input	Output			
$x = (x_1, x_2, x_3, x_4, x_5)$	g_1	g_2	g_3	g_4 $(= \widetilde{\mathbb{C}}(x))$
$x^{(1)} = (0, 0, 0, 0, 0)$	y_1	y_2	y_3	0

If A obtains 1 for a query $x^{(2)} := (0, 0, 0, 0, 1)$, it indicates that the output was flipped by the gate g_2 since only the input to g_2 was modified. Therefore, the truth table must be updated as follows:

Input	Output			
$x = (x_1, x_2, x_3, x_4, x_5)$	g_1	g_2	g_3	g_4 $(= \widetilde{\mathbb{C}}(x))$
$x^{(1)} = (0, 0, 0, 0, 0)$	y_1	y_2	y_3	0
$x^{(2)} = (0, 0, 0, 0, 1)$	y_1	y_2	$\overline{y_3}$	1

As such, A obtains a relation between inputs and the output of g_4; it outputs 0 for the input (y_2, y_3) and outputs 1 for $(y_2, \overline{y_3})$. Suppose that A makes two more queries, $x^{(3)} := (0, 0, 1, 0, 0)$ and $x^{(4)} := (0, 0, 1, 0, 1)$, and obtains 1 and 1, respectively. Since $g_3(0, 0) = y_3$ holds from previous queries, A can infer that g_2 outputs $\overline{y_2}$ based on the observation that the output of $\widetilde{\mathbb{C}}$ changes between $x^{(1)}$ and $x^{(3)}$. Then, the updated truth table must be as follows.

Input	Output			
$x = (x_1, x_2, x_3, x_4, x_5)$	g_1	g_2	g_3	g_4 $(= \widetilde{\mathbb{C}}(x))$
$x^{(1)} = (0,0,0,0,0)$	y_1	y_2	y_3	0
$x^{(2)} = (0,0,0,0,1)$	y_1	y_2	$\overline{y_3}$	1
$x^{(3)} = (0,0,1,0,0)$	y_1	$\overline{y_2}$	y_3	1
$x^{(4)} = (0,0,1,0,1)$	y_1	$\overline{y_2}$	$\overline{y_3}$	1

As a result, A learns all input-output patterns of g_4 using just four queries. At this point, A already knows two input-output patterns for each of g_2 and g_3, respectively. Specifically, A has determined $g_2(y_1, 0) = y_2$, $g_2(y_1, 1) = \overline{y_2}$, $g_3(0,0) = y_3$, and $g_3(0,1) = \overline{y_3}$. Thus, two additional queries are required to learn all remaining input-output patterns for both g_2 and g_3. Importantly, this learning process cannot be parallelized, since each input-output pattern is inferred by observing the output difference between the current and previous queries. For instance, to determine the outputs of $g_2(\overline{y_1}, 0)$ and $g_2(\overline{y_1}, 1)$, A must construct queries where the inputs to other gates do not affect the output of g_3. In this case, A must fix the some inputs of a query as either $x = (\cdot, \cdot, \cdot, 0, 0)$ or $x = (\cdot, \cdot, \cdot, 0, 1)$, ensuring that the output of g_3 remains unchanged from previous queries. Otherwise, if the inputs to g_3 are not fixed appropriately, A cannot accurately observe the difference related to the output of g_2 between the current and previous queries, as the outputs of $g_3(1,0)$ or $g_3(1,1)$ remain unknown.

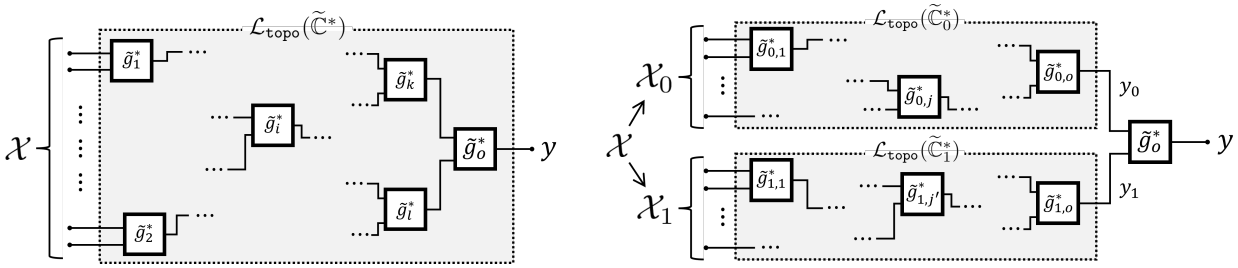
Let us show a concrete example. Suppose that A makes two queries $x^{(5)} \coloneqq (1,1,1,0,0)$ and $x^{(6)} \coloneqq (1,1,1,0,1)$ to learn the remaining input-output patterns of g_3, i.e., to determine the output of g_3 for the input $(\overline{y_1}, x_3 = 0)$ or $(\overline{y_1}, x_3 = 1)$. Then, the updated truth table is given as follows.

Input	Output			
$x = (x_1, x_2, x_3, x_4, x_5)$	g_1	g_2	g_3	g_4 $(= \widetilde{\mathbb{C}}(x))$
$x^{(1)} = (0,0,0,0,0)$	y_1	y_2	y_3	0
$x^{(2)} = (0,0,0,0,1)$	y_1	y_2	$\overline{y_3}$	1
$x^{(3)} = (0,0,1,0,0)$	y_1	$\overline{y_2}$	y_3	1
$x^{(4)} = (0,0,1,0,1)$	y_1	$\overline{y_2}$	$\overline{y_3}$	1
$x^{(5)} = (1,1,1,0,0)$	$\overline{y_1}$	y_2	y_3	0
$x^{(6)} = (1,1,0,0,0)$	$\overline{y_1}$	y_2	y_3	0

Clearly, A now learns $g_2(\overline{y_1}, 0) = y_2$ and $g_2(\overline{y_1}, 1) = y_2$ based on the following observations. First, comparing the outputs of $x^{(3)}$ and $x^{(5)}$, A sees a change that results solely from modifying the first two input bits, i.e., the inputs to g_1. This implies that the output of g_1 is flipped when its input changes from $(0,0)$ to $(1,1)$, leading to the conclusion that $g_2(\overline{y_1}, 1) = y_2$. Similarly, $g_2(\overline{y_1}, 0) = y_2$ can be determined by analyzing the output difference between $x^{(5)}$ and $x^{(6)}$. Since $\widetilde{\mathbb{C}}(x^{(5)}) = \widetilde{\mathbb{C}}(x^{(6)})$, and prior queries have already established $g_1(1,1) = \overline{y_1}$ and $g_3(0,0) = y_3$, the observed difference must result from a change in g_2's input.

In a similar manner, two additional queries are needed to determine the remaining input-output patterns for each of g_1 and g_3. Therefore, a total of 10 queries are required to recover all input-output patterns of the entire circuit $\widetilde{\mathbb{C}}$. Once these patterns are known, A can compute $\widetilde{\mathbb{C}}(x)$ for any $x \in \{0,1\}^{\ell_i}$, effectively reconstructing the entire functionality of the circuit.

The above observation can be generalized to any ℓ_i-input ℓ_o-output circuits $\mathbb{C}$. As discussed earlier, such a circuit can be decomposed into ℓ_o fan-in cones $\widetilde{\mathbb{C}}_1, \widetilde{\mathbb{C}}_2, \ldots, \widetilde{\mathbb{C}}_{\ell_o}$ of $\mathbb{C}$, and a single oracle query might be used to analyze all of them in parallel. Therefore, we focus on the largest sub-circuit $\widetilde{\mathbb{C}}^* \in \{\widetilde{\mathbb{C}}_i\}_{i=1}^{\ell_o}$, defined as the sub-circuit containing the most gates among all sub-circuits. Let $\widetilde{\ell}_g^*$ denote the number of gates in $\widetilde{\mathbb{C}}^*$. As demonstrated earlier, A requires at least four queries



(a) Topology $\mathcal{L}_{\mathtt{topo}}(\widetilde{\mathbb{C}}^*)$ **(b)** Equivalent variant of $\mathcal{L}_{\mathtt{topo}}(\widetilde{\mathbb{C}}^*)$

Figure 6: Topologies of the largest fan-in cone $\widetilde{\mathbb{C}}^*$ and its equivalent variant that consists of an output gate $\widetilde{g}_o^*$ and two sub-fan-in cones $\widetilde{\mathbb{C}}_0^*$ and $\widetilde{\mathbb{C}}_1^*$, where $\mathcal{X} = \mathcal{X}_0 \cup \mathcal{X}_1$. Note that it clearly hold $\widetilde{g}_k^* = \widetilde{g}_{0,o}^*$ and $\widetilde{g}_l^* = \widetilde{g}_{1,o}^*$ in this example. Furthermore, $\widetilde{\mathbb{C}}_0^*$ and $\widetilde{\mathbb{C}}_1^*$ might share some inputs and gates; namely, it might hold $\mathcal{X}_0 \cap \mathcal{X}_1 \neq \emptyset$ and, for instance, $\widetilde{g}_i^* = \widetilde{g}_{0,j}^* = \widetilde{g}_{1,j'}^*$.

to determine all input-output patterns of the output gate of $\widetilde{\mathbb{C}}^*$, where the output gate refers to the gate that produces the output value $\widetilde{\mathbb{C}}^*(x)$. For each of the remaining $(\widetilde{\ell}_g^* - 1)$ gates, an additional two queries per gate are needed to fully determine the functionality of $\widetilde{\mathbb{C}}^*$. Consequently, the minimum number of queries required to recover all input-output patterns of the sub-circuit $\widetilde{\mathbb{C}}^*$, and thus of the entire circuit $\mathbb{C}$, is $4 + 2(\widetilde{\ell}_g^* - 1) = 2\widetilde{\ell}_g^* + 2$. This is the intuition of the proof.

Let us now formalize this intuition. A key observation is that the fan-in cone $\widetilde{\mathbb{C}}^*$ consists of a single output gate g_o^* and two *sub-fan-in cones*, denoted $\widetilde{\mathbb{C}}_0^*$ and $\widetilde{\mathbb{C}}_1^*$, as illustrated in Figure 6 in terms of their topological structure. For instance, in the concrete example in Figure 5, $\widetilde{\mathbb{C}}_0^*$ and $\widetilde{\mathbb{C}}_1^*$ consist of $\{g_1, g_2\}$ and $\{g_3\}$, respectively, and $g_o^* := g_4$. Let $\mathcal{X}$ be an (ordered) set of input bits to $\widetilde{\mathbb{C}}^*$, namely, $\mathcal{X} = \{x_1, x_2, \ldots, x_{\ell_i}\}$ for a query $x = (x_1, x_2, \ldots, x_{\ell_i})$. Similarly, let $\mathcal{X}_0$ and $\mathcal{X}_1$ be (ordered) sets of input bits to $\widetilde{\mathbb{C}}_0^*$ and $\widetilde{\mathbb{C}}_1^*$, respectively. Obviously, it hold that $\mathcal{X}_0, \mathcal{X}_1 \subset \mathcal{X}$ and $\mathcal{X} = \mathcal{X}_0 \cup \mathcal{X}_1$.

Suppose a powerful adversary $\mathsf{A}^\star$ that always follows the optimal strategy, i.e., it can choose the most informative queries x (or $\mathcal{X}$) to recover $\widetilde{\mathbb{C}}^*$. Such an adversary must exist, as we consider all possible adversaries. As shown earlier, the only way to fully and uniquely recover $\widetilde{\mathbb{C}}^*$ from its topology $\mathcal{L}_{\mathtt{topo}}(\widetilde{\mathbb{C}}^*)$ with oracle accesses is, as demonstrated above, to determine the relationships between the input-output pairs obtained from oracle queries and all the internal wires in $\mathcal{L}_{\mathtt{topo}}(\widetilde{\mathbb{C}}^*)$. To achieve this, the first step must be to determine the functional relationship between the two input wires of the output gate $\widetilde{g}_o^*$ and its output, i.e., the output of $\widetilde{\mathbb{C}}^*$. It is obvious that at least four queries are required for that purpose since, in general, four queries $(0,0)$, $(0,1)$, $(1,0)$, and $(1,1)$ are necessary even for determining outputs of a single gate. Indeed, $\mathsf{A}^\star$ only requires four queries to determine the relationships between the input-output pairs and input wires of $\widetilde{g}_o^*$ as follows.

For any first query $\mathcal{X}^{(1)} = (\mathcal{X}_0^{(1)}, \mathcal{X}_1^{(1)})$,[4] let $y_0 := \widetilde{\mathbb{C}}_0^*(\mathcal{X}_0^{(1)})$, $y_1 := \widetilde{\mathbb{C}}_1^*(\mathcal{X}_1^{(1)})$, and $y^{(1)} := \widetilde{g}_o^*(y_0, y_1) = \widetilde{\mathbb{C}}^*(\mathcal{X})$, as a baseline (see also Figure 6(b)). Then, a truth table with the variables y_0, y_1, and $y^{(1)}$ is as follows:

Input	Output		
$\mathcal{X} = (\mathcal{X}_0, \mathcal{X}_1)$	$\widetilde{\mathbb{C}}_0^*(\mathcal{X}_0)$	$\widetilde{\mathbb{C}}_1^*(\mathcal{X}_1)$	$\widetilde{g}_o^*$ $(= \widetilde{\mathbb{C}}^*(\mathcal{X}))$
$\mathcal{X}^{(1)} = (\mathcal{X}_0^{(1)}, \mathcal{X}_1^{(1)})$	y_0	y_1	$y^{(1)}$

Note that $\mathsf{A}^\star$ can observe only $y^{(1)}$ and has no visibility into the internal values of y_0 and y_1. Now,

[4] Although the notation $\mathcal{X} = (\mathcal{X}_0, \mathcal{X}_1)$ is not mathematically rigorous, we adopt it for clarity and ease of explanation.

$\mathsf{A}^\star$ knows the output $y^{(1)}$ of $\widetilde{g}_o^*$ for the input (y_0, y_1). Therefore, the first step is completed if $\mathsf{A}^\star$ determines the output values of $\widetilde{g}_o^*$ for inputs $(\overline{y_0}, y_1)$, $(y_0, \overline{y_1})$, and $(\overline{y_0}, \overline{y_1})$. The only way to achieve this is to observe what query changes the output value of $\widetilde{g}_o^*$. There are the following facts:

- There must be at least one pair of *flipped wires* of $\widetilde{g}_o^*$, which is input wires that yield $\overline{y}^{(1)}$. Namely, at least one output among $\widetilde{g}_o^*(\overline{y_0}, y_1)$, $\widetilde{g}_o^*(y_0, \overline{y_1})$, and $\widetilde{g}_o^*(\overline{y_0}, \overline{y_1})$ must be $\overline{y}^{(1)}$. Otherwise, $\widetilde{g}_o^*$ always outputs the same value $y^{(1)}$ (0 or 1) for any input wires, which is meaningless.

Based on this fact, suppose that $\mathsf{A}^\star$ chooses the second query $\mathcal{X}^{(2)}$ that yields the flipped wires of $\widetilde{g}_o^*$ (and $\mathsf{A}^\star$ can do so by assumption). Without loss of generality, we here assume that $(\overline{y_0}, y_1)$ is the flipped wires. Therefore, the updated truth table is as follows.

Input	Output		
$\mathcal{X} = (\mathcal{X}_0, \mathcal{X}_1)$	$\widetilde{\mathbb{C}}_0^*(\mathcal{X}_0)$	$\widetilde{\mathbb{C}}_1^*(\mathcal{X}_1)$	$\widetilde{g}_o^*$ $(= \widetilde{\mathbb{C}}^*(\mathcal{X}))$
$\mathcal{X}^{(1)} = (\mathcal{X}_0^{(1)}, \mathcal{X}_1^{(1)})$	y_0	y_1	$y^{(1)}$
$\mathcal{X}^{(2)} = (\mathcal{X}_0^{(2)}, \mathcal{X}_1^{(2)})$	$\overline{y_0}$	y_1	$y^{(2)} = \overline{y}^{(1)}$

Again, $\mathsf{A}^\star$ only observes the output value of $\widetilde{\mathbb{C}}^*(\mathcal{X}^{(2)})$, i.e., $y^{(2)}$. Hence, $\mathsf{A}^\star$ understands that the flipped wires are $(\overline{y_0}, y_1)$ if and only if $\mathcal{X}_0^{(1)} \neq \mathcal{X}_0^{(2)}$ and $\mathcal{X}_1^{(1)} = \mathcal{X}_1^{(2)}$ hold. In other words, $\mathsf{A}^\star$ has to fix the input to $\widetilde{\mathbb{C}}_1^*$ since $\mathcal{X}_1^{(1)} = \mathcal{X}_1^{(2)}$ ensures the output of y_1. Otherwise, i.e., if $\mathcal{X}_1^{(1)} \neq \mathcal{X}_1^{(2)}$ holds, $\mathsf{A}^\star$ cannot determine the flipped wires are $(y_0, \overline{y_1})$, $(\overline{y_0}, y_1)$, or $(\overline{y_0}, \overline{y_1})$, since it is unclear to $\mathsf{A}^\star$ that $\widetilde{\mathbb{C}}_1^*(\mathcal{X}_1^{(2)})$ outputs y_1 or $\overline{y_1}$. Therefore, we may rewrite the above truth table as follows:

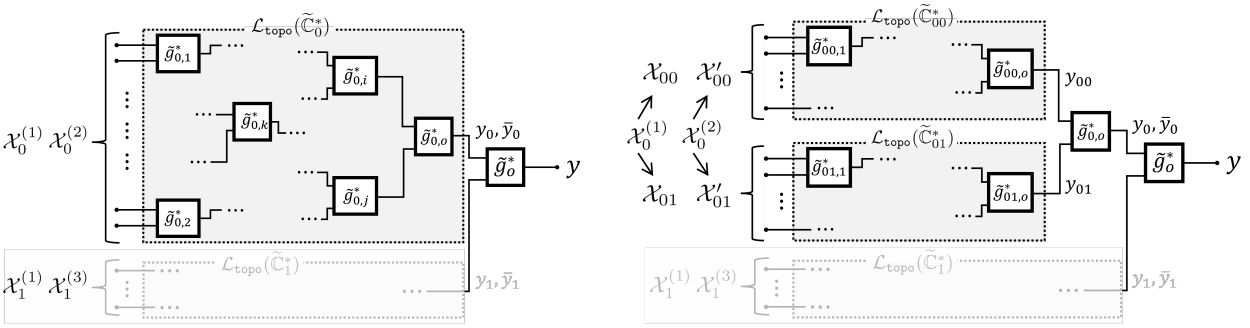
Input	Output		
$\mathcal{X} = (\mathcal{X}_0, \mathcal{X}_1)$	$\widetilde{\mathbb{C}}_0^*(\mathcal{X}_0)$	$\widetilde{\mathbb{C}}_1^*(\mathcal{X}_1)$	$\widetilde{g}_o^*$ $(= \widetilde{\mathbb{C}}^*(\mathcal{X}))$
$\mathcal{X}^{(1)} = (\mathcal{X}_0^{(1)}, \mathcal{X}_1^{(1)})$	y_0	y_1	$y^{(1)}$
$\mathcal{X}^{(2)} = (\mathcal{X}_0^{(2)}, \mathcal{X}_1^{(1)})$	$\overline{y_0}$	y_1	$y^{(2)} = \overline{y}^{(1)}$

Next, we assume that $(y_0, \overline{y_1})$ is also flipped wires without loss of generality. $\mathsf{A}^\star$ can find appropriate $\mathcal{X}^{(3)} = (\mathcal{X}_0^{(3)}, \mathcal{X}_1^{(3)})$ that yields $(y_0, \overline{y_1})$ and $y^{(3)} = \overline{y}^{(1)}$, where $\mathcal{X}_0^{(1)} = \mathcal{X}_0^{(3)}$ and $\mathcal{X}_1^{(1)} \neq \mathcal{X}_1^{(3)}$. More specifically, since it holds $\mathcal{X}_0^{(1)} = \mathcal{X}_0^{(3)}$, we have $\widetilde{\mathbb{C}}_0^*(\mathcal{X}_0^{(3)}) = \widetilde{\mathbb{C}}_0^*(\mathcal{X}_0^{(1)}) = y_0$. Therefore, $\widetilde{\mathbb{C}}_1^*(\mathcal{X}_1^{(3)})$ must output $\overline{y_1}$ to consistently satisfy $y^{(3)} = \overline{y}^{(1)}$. Hence, the updated truth table is as follows:

Input	Output		
$\mathcal{X} = (\mathcal{X}_0, \mathcal{X}_1)$	$\widetilde{\mathbb{C}}_0^*(\mathcal{X}_0)$	$\widetilde{\mathbb{C}}_1^*(\mathcal{X}_1)$	$\widetilde{g}_o^*$ $(= \widetilde{\mathbb{C}}^*(\mathcal{X}))$
$\mathcal{X}^{(1)} = (\mathcal{X}_0^{(1)}, \mathcal{X}_1^{(1)})$	y_0	y_1	$y^{(1)}$
$\mathcal{X}^{(2)} = (\mathcal{X}_0^{(2)}, \mathcal{X}_1^{(1)})$	$\overline{y_0}$	y_1	$y^{(2)} = \overline{y}^{(1)}$
$\mathcal{X}^{(3)} = (\mathcal{X}_0^{(1)}, \mathcal{X}_1^{(3)})$	y_0	$\overline{y_1}$	$y^{(3)} = \overline{y}^{(1)}$

Finally, $\mathsf{A}^\star$ sets a fourth query $\mathcal{X}^{(4)} = (\mathcal{X}_0^{(4)}, \mathcal{X}_1^{(4)}) := (\mathcal{X}_0^{(2)}, \mathcal{X}_1^{(3)})$. Since $\mathsf{A}^\star$ already knows $\widetilde{\mathbb{C}}_0^*(\mathcal{X}_0^{(2)}) = \overline{y_0}$ and $\widetilde{\mathbb{C}}_1^*(\mathcal{X}_1^{(3)}) = \overline{y_1}$, $\mathcal{X}^{(4)}$ yields $(\overline{y_0}, \overline{y_1})$. Therefore, the truth table is updated below:

Input	Output		
$\mathcal{X} = (\mathcal{X}_0, \mathcal{X}_1)$	$\widetilde{\mathbb{C}}_0^*(\mathcal{X}_0)$	$\widetilde{\mathbb{C}}_1^*(\mathcal{X}_1)$	$\widetilde{g}_o^*$ $(= \widetilde{\mathbb{C}}^*(\mathcal{X}))$
$\mathcal{X}^{(1)} = (\mathcal{X}_0^{(1)}, \mathcal{X}_1^{(1)})$	y_0	y_1	$y^{(1)}$
$\mathcal{X}^{(2)} = (\mathcal{X}_0^{(2)}, \mathcal{X}_1^{(1)})$	$\overline{y_0}$	y_1	$y^{(2)} = \overline{y}^{(1)}$
$\mathcal{X}^{(3)} = (\mathcal{X}_0^{(1)}, \mathcal{X}_1^{(3)})$	y_0	$\overline{y_1}$	$y^{(3)} = \overline{y}^{(1)}$
$\mathcal{X}^{(4)} = (\mathcal{X}_0^{(2)}, \mathcal{X}_1^{(3)})$	$\overline{y_0}$	$\overline{y_1}$	$y^{(4)}$



(a) Topology $\mathcal{L}_{\mathsf{topo}}(\widetilde{\mathbb{C}}_0^*)$ in $\mathcal{L}_{\mathsf{topo}}(\widetilde{\mathbb{C}}^*)$ **(b)** Equivalent variant of $\mathcal{L}_{\mathsf{topo}}(\widetilde{\mathbb{C}}_0^*)$

Figure 7: Topologies of the sub-fan-in cone $\widetilde{\mathbb{C}}_0^*$ (in $\widetilde{\mathbb{C}}^*$) and its equivalent variant that consists of an output gate $\widetilde{g}_{0,o}^*$ and two sub-fan-in cones $\widetilde{\mathbb{C}}_{00}^*$ and $\widetilde{\mathbb{C}}_{01}^*$. Note that it hold that $\mathcal{X}_0^* = \mathcal{X}_{00}^* \cup \mathcal{X}_{01}^*$ and $\overline{\mathcal{X}}_0^* = \overline{\mathcal{X}}_{00}^* \cup \overline{\mathcal{X}}_{01}^*$.

At the end of the first step, $\mathsf{A}^\star$ knows that (a) $\widetilde{\mathbb{C}}_0^*(\mathcal{X}_0^{(1)}) = y_0$, (b) $\widetilde{\mathbb{C}}_0^*(\mathcal{X}_0^{(2)}) = \overline{y_0}$, (c) $\widetilde{\mathbb{C}}_1^*(\mathcal{X}_1^{(1)}) = y_1$, (d) $\widetilde{\mathbb{C}}_1^*(\mathcal{X}_1^{(3)}) = \overline{y_1}$, and (e) all input-output pairs for $\widetilde{g}_o^*$, say, $((y_0, y_1), y^{(1)})$. Although we have shown the case that $(y_0, \overline{y_1})$ and $(\overline{y_0}, y_1)$ are flipped wires, one may check that the same holds or more queries are required for other cases.[5]

The second step is done recursively, but cannot be parallelized. Specifically, each sub-fan-in cone can then be similarly decomposed into its output gate and two *sub-sub-fan-in cones*, and an additional two queries per the output gate are needed to learn the relationships between the sub-sub-fan-in cones and the output gate. For instance $\widetilde{\mathbb{C}}_0^*$ can also be viewed as a combination of an output gate $\widetilde{g}_{0,o}^*$ and two sub-sub-fan-in cones, $\widetilde{\mathbb{C}}_{00}^*$ and $\widetilde{\mathbb{C}}_{01}^*$ (see Figure 7). As mentioned above, this process cannot be parallelized, as the inputs to $\widetilde{\mathbb{C}}_1^*$ must be fixed when analyzing $\widetilde{\mathbb{C}}_0^*$, consisting of $\widetilde{g}_{0,o}^*$ and $(\widetilde{\mathbb{C}}_{00}^*, \widetilde{\mathbb{C}}_{01}^*)$, and vice versa. Otherwise, $\mathsf{A}^\star$ cannot determine which inputs to the sub-fan-in cones caused the output of the circuit $\widetilde{\mathbb{C}}^*$ to flip. Consequently, a recursive, step-by-step analysis is required to fully recover the circuit, leading directly to the established query bound.

Let us look into the second step with the case for $\widetilde{\mathbb{C}}_0^*$ without loss of generality. $\mathsf{A}^\star$ now has the two input-output pair, $(\mathcal{X}_0^{(1)}, y_0)$ and $(\mathcal{X}_0^{(2)}, \overline{y_0})$, for the sub-fan-in cone $\widetilde{\mathbb{C}}_0^*$. More specifically for the equivalent variant in Figure 7(b), according to the sub-sub-fan-in cones $\widetilde{\mathbb{C}}_{00}^*$ and $\widetilde{\mathbb{C}}_{01}^*$, the inputs $\mathcal{X}_0^{(1)}$ and $\mathcal{X}_0^{(2)}$ can be separated into $(\mathcal{X}_{00}, \mathcal{X}_{01})$ and $(\mathcal{X}'_{00}, \mathcal{X}'_{01})$, respectively, where $\mathcal{X}_{00}, \mathcal{X}_{01} \subset \mathcal{X}_0^{(1)}$, $\mathcal{X}'_{00}, \mathcal{X}'_{01} \subset \mathcal{X}_0^{(2)}$, $\mathcal{X}_0^{(1)} = \mathcal{X}_{00} \cup \mathcal{X}_{01}$, and $\mathcal{X}_0^{(2)} = \mathcal{X}'_{00} \cup \mathcal{X}'_{01}$. Let $y_{00} \coloneqq \widetilde{\mathbb{C}}_{00}^*(\mathcal{X}_{00})$, $y_{01} \coloneqq \widetilde{\mathbb{C}}_{01}^*(\mathcal{X}_{01})$ as a baseline. Then, the values of $y'_{00} \coloneqq \widetilde{\mathbb{C}}_{00}^*(\mathcal{X}'_{00})$, $y'_{01} \coloneqq \widetilde{\mathbb{C}}_{01}^*(\mathcal{X}'_{01})$ may vary depending on the relationships between $\mathcal{X}_0^{(1)} = (\mathcal{X}_{00}, \mathcal{X}_{01})$ and $\mathcal{X}_0^{(2)} = (\mathcal{X}'_{00}, \mathcal{X}'_{01})$; since it holds $\mathcal{X}_0^{(1)} = (\mathcal{X}_{00}, \mathcal{X}_{01}) \neq (\mathcal{X}'_{00}, \mathcal{X}'_{01}) = \mathcal{X}_0^{(2)}$, there are the following three cases:

(i) Case of $(\mathcal{X}_{00} = \mathcal{X}'_{00})$ and $(\mathcal{X}_{01} \neq \mathcal{X}'_{01})$: y'_{00} should satisfy $y'_{00} = \widetilde{\mathbb{C}}_{00}^*(\mathcal{X}'_{00}) = \widetilde{\mathbb{C}}_{00}^*(\mathcal{X}_{00}) = y_{00}$. Then, since the output of $\widetilde{\mathbb{C}}_0^*$ is flipped from y_0 to $\overline{y_0}$, we have $y'_{01} = \overline{y_{01}}$.

(ii) Case of $(\mathcal{X}_{00} \neq \mathcal{X}'_{00})$ and $(\mathcal{X}_{01} = \mathcal{X}'_{01})$: Similar to the above, y'_{01} should satisfy $y'_{01} = \widetilde{\mathbb{C}}_{01}^*(\mathcal{X}'_{01}) = \widetilde{\mathbb{C}}_{01}^*(\mathcal{X}_{01}) = y_{01}$, and we have $y_{00} = \overline{y_{00}}$ since the output of $\widetilde{\mathbb{C}}_0^*$ is flipped from y_0 to $\overline{y_0}$.

[5] $\mathsf{A}^\star$ may make more queries at the first step, and $\mathsf{A}^\star$ can get other relations, say, $\widetilde{\mathbb{C}}_0^*(\mathcal{X}_0^{(5)}) = y_0$ from a fifth query $\mathcal{X}^{(5)} = (\mathcal{X}_0^{(5)}, \mathcal{X}_1^{(5)})$. However, it does not increase $\mathsf{A}^\star$'s knowledge or help to accelerate the next steps.

(iii) Case of $(\mathcal{X}_{00} \neq \mathcal{X}'_{00})$ and $(\mathcal{X}_{01} \neq \mathcal{X}'_{01})$: $\mathsf{A}^\star$ cannot get to know the corresponding flipped wires (y'_{00}, y'_{01}) since all input-wire patterns, i.e., (y_{00}, y_{01}), $(\overline{y_{00}}, y_{01})$, $(y_{00}, \overline{y_{01}})$, $(\overline{y_{00}}, \overline{y_{01}})$ can satisfy the condition.

Since we consider the worst-case scenario with $\mathsf{A}^\star$, we here assume the case (i) without loss of generality. Then, the truth table can be restated as follows:[6]

Input $\mathcal{X} = (\mathcal{X}_0 = (\mathcal{X}_{00}, \mathcal{X}_{01}), \mathcal{X}_1)$	Output $\widetilde{\mathbb{C}}^*_{00}(\mathcal{X}_{00})$	$\widetilde{\mathbb{C}}^*_{01}(\mathcal{X}_{01})$	$\widetilde{\mathbb{C}}^*_0(\mathcal{X}_0)$	$\widetilde{\mathbb{C}}^*_1(\mathcal{X}_1)$	$\widetilde{g}^*_o\ (= \widetilde{\mathbb{C}}^*(\mathcal{X}))$
$\mathcal{X}^{(1)} = (\mathcal{X}_0^{(1)} = (\mathcal{X}_{00}, \mathcal{X}_{01}), \mathcal{X}_1^{(1)})$	y_{00}	y_{01}	y_0	y_1	$y^{(1)}$
$\mathcal{X}^{(2)} = (\mathcal{X}_0^{(2)} = (\mathcal{X}_{00}, \mathcal{X}'_{01}), \mathcal{X}_1^{(1)})$	y_{00}	$\overline{y_{01}}$	$\overline{y_0}$	y_1	$y^{(2)} = \overline{y}^{(1)}$
$\mathcal{X}^{(3)} = (\mathcal{X}_0^{(1)} = (\mathcal{X}_{00}, \mathcal{X}_{01}), \mathcal{X}_1^{(3)})$	y_{00}	y_{01}	y_0	$\overline{y_1}$	$y^{(3)} = \overline{y}^{(1)}$
$\mathcal{X}^{(4)} = (\mathcal{X}_0^{(2)} = (\mathcal{X}_{00}, \mathcal{X}'_{01}), \mathcal{X}_1^{(3)})$	y_{00}	$\overline{y_{01}}$	$\overline{y_0}$	$\overline{y_1}$	$y^{(4)}$

It is clear that $\mathcal{X}^{(3)}$ and $\mathcal{X}^{(4)}$ provide no additional information on $\widetilde{\mathbb{C}}^*_0$ beyond what $\mathcal{X}^{(1)}$ and $\mathcal{X}^{(2)}$ reveal, since $\mathcal{X}^{(3)}$ (resp., $\mathcal{X}^{(4)}$) shares the same values related to $\widetilde{\mathbb{C}}^*_0$ as $\mathcal{X}^{(1)}$ (resp., $\mathcal{X}^{(2)}$). Thus, any bit flips in $\widetilde{\mathbb{C}}^*(\mathcal{X}^{(3)})$ or $\widetilde{\mathbb{C}}^*(\mathcal{X}^{(4)})$ must be attributed to the values associated with $\widetilde{\mathbb{C}}^*_1$. Therefore, $\mathsf{A}^\star$ already knows two input-output pairs for $\widetilde{\mathbb{C}}^*_0$, $((y_{00}, y_{01}), y_0)$ and $((y_{00}, \overline{y_{01}}), \overline{y_0})$, and needs to know outputs corresponding to other inputs, $(\overline{y_{00}}, y_{01})$ and $(\overline{y_{00}}, \overline{y_{01}})$, requiring two additional queries.

Suppose that $\mathsf{A}^\star$ makes two queries $\mathcal{X}^{(5)} = (\mathcal{X}_0^{(5)} = (\mathcal{X}''_{00}, \mathcal{X}_{01}), \mathcal{X}_1^{(5)} = \mathcal{X}_1^{(1)})$ and $\mathcal{X}^{(6)} = (\mathcal{X}_0^{(6)} = (\mathcal{X}''_{00}, \mathcal{X}'_{01}), \mathcal{X}_1^{(6)} = \mathcal{X}_1^{(1)})$, and obtains $y^{(5)} = \overline{y}^{(1)}$ and $y^{(6)} = y^{(1)}$ as the output of the fan-in cone $\widetilde{\mathbb{C}}^*$, respectively. The only difference between $\mathcal{X}^{(1)}$ and $\mathcal{X}^{(5)}$ is $\mathcal{X}_{00}$ and $\mathcal{X}''_{00}$. Considering that the corresponding outputs are also different, i.e., $y^{(5)} = \overline{y}^{(1)}$, we have $\widetilde{\mathbb{C}}^*_{00}(\mathcal{X}''_{00}) = \overline{y_{00}}$ and hence $\widetilde{\mathbb{C}}^*_0(\mathcal{X}''_{00}, \mathcal{X}_{01}) = \overline{y_0}$. $\mathsf{A}^\star$ then knows $\widetilde{\mathbb{C}}^*_{00}(\mathcal{X}''_{00}) = \overline{y_{00}}$ and $\widetilde{\mathbb{C}}^*_{01}(\mathcal{X}'_{01}) = \overline{y_{01}}$, and therefore, can determine $\widetilde{\mathbb{C}}^*_0(\mathcal{X}''_{00}, \mathcal{X}'_{01}) = y_0$ since the output $y^{(6)}$ is flipped from $y^{(5)}$. In summary, the updated truth table is as follows:

Input $\mathcal{X} = (\mathcal{X}_0 = (\mathcal{X}_{00}, \mathcal{X}_{01}), \mathcal{X}_1)$	Output $\widetilde{\mathbb{C}}^*_{00}(\mathcal{X}_{00})$	$\widetilde{\mathbb{C}}^*_{01}(\mathcal{X}_{01})$	$\widetilde{\mathbb{C}}^*_0(\mathcal{X}_0)$	$\widetilde{\mathbb{C}}^*_1(\mathcal{X}_1)$	$\widetilde{g}^*_o\ (= \widetilde{\mathbb{C}}^*(\mathcal{X}))$
$\mathcal{X}^{(1)} = (\mathcal{X}_0^{(1)} = (\mathcal{X}_{00}, \mathcal{X}_{01}), \mathcal{X}_1^{(1)})$	y_{00}	y_{01}	y_0	y_1	$y^{(1)}$
$\mathcal{X}^{(2)} = (\mathcal{X}_0^{(2)} = (\mathcal{X}_{00}, \mathcal{X}'_{01}), \mathcal{X}_1^{(1)})$	y_{00}	$\overline{y_{01}}$	$\overline{y_0}$	y_1	$y^{(2)} = \overline{y}^{(1)}$
$\mathcal{X}^{(3)} = (\mathcal{X}_0^{(1)} = (\mathcal{X}_{00}, \mathcal{X}_{01}), \mathcal{X}_1^{(3)})$	y_{00}	y_{01}	y_0	$\overline{y_1}$	$y^{(3)} = \overline{y}^{(1)}$
$\mathcal{X}^{(4)} = (\mathcal{X}_0^{(2)} = (\mathcal{X}_{00}, \mathcal{X}'_{01}), \mathcal{X}_1^{(3)})$	y_{00}	$\overline{y_{01}}$	$\overline{y_0}$	$\overline{y_1}$	$y^{(4)}$
$\mathcal{X}^{(5)} = (\mathcal{X}_0^{(5)} = (\mathcal{X}''_{00}, \mathcal{X}_{01}), \mathcal{X}_1^{(1)})$	$\overline{y_{00}}$	y_{01}	$\overline{y_0}$	y_1	$y^{(5)} = \overline{y}^{(1)}$
$\mathcal{X}^{(6)} = (\mathcal{X}_0^{(6)} = (\mathcal{X}''_{00}, \mathcal{X}'_{01}), \mathcal{X}_1^{(1)})$	$\overline{y_{00}}$	$\overline{y_{01}}$	y_0	y_1	$y^{(6)} = y^{(1)}$

We emphasize again that the inputs to $\widetilde{\mathbb{C}}^*_1$ must be fixed during the analysis of $\widetilde{\mathbb{C}}^*_0$, making the second step inherently sequential. This step is then repeated for $\widetilde{\mathbb{C}}^*_1$, followed by recursive decomposition of the sub-sub-fan-in cones $\widetilde{\mathbb{C}}^*_b$ ($b \in \{0,1\}^2$), and so on. At each recursion level, the adversary has two input-output patterns and must issue two additional queries. Since there are $\widetilde{\ell}^*_g - 1$ recursive steps, the total number of queries required to recover all input-output patterns of the circuit $\mathbb{C}$ is $4 + 2(\widetilde{\ell}^*_g - 1) = 2\widetilde{\ell}^*_g + 2$.

It is important to note that, since (t, γ, η)-CRA resilience is defined with respect to *any* t-time adversary, there must be an adversary $\mathsf{A}^\star$ that follows the ideal strategy described above, namely, one that issues the minimum number of queries necessary to fully recover the sub-circuit.

[6] We gray out the entries related to $\widetilde{\mathbb{C}}^*_1$ in the truth table to emphasize the values associated with $\widetilde{\mathbb{C}}^*_0$.

Furthermore, under the practical assumption that the circuit size ℓ is polynomially bounded, the number of queries (and consequently $\mathsf{A}^\star$'s running time t) remains polynomial in ℓ.

The remaining factor we must address is the relaxation parameter $\gamma \in (0, 1]$. Clearly, the adversary $\mathsf{A}^\star$ described above can fully recover the functionality of the circuit $\mathbb{C}$ using at least $2\widetilde{\ell}_g^* + 2$ queries. Conversely, with fewer than $2\widetilde{\ell}_g^* + 2$ queries, $\mathsf{A}^\star$ cannot determine any input-output pattern that A has not explicitly queried. Therefore, if γ is small enough, specifically, if $\gamma \cdot 2^{\ell_i} < 2\widetilde{\ell}_g^* + 2$, then A can achieve its goal of recovering only $\gamma \cdot 2^{\ell_i}$ input-output patterns by making exactly $\gamma \cdot 2^{\ell_i}$ queries.

It completes the proof. □

4.4 Discussions

On the Power of Real-world Adversaries. Theorems 2 and 3 analyze the minimum number of oracle queries required for successful attacks. While the lower bound in Theorem 3 grows linearly with the size of the largest fan-in cone $\widetilde{\mathbb{C}}^*$ and may appear weak, it holds unconditionally, even against computationally unbounded adversaries. This reflects the fact that our KRA/CRA resilience definitions (Definitions 6 and 7) are intended to capture *worst-case scenarios*, representing the strongest possible attacks under the given leakage $\mathcal{L}$.

Importantly, the existence of a powerful adversary $\mathsf{A}^\star$ in theory does not imply its practical realizability; there remains a substantial gap between such ideal adversaries and real-world ones, such as those using SAT solvers. The bound theoretically strengthens how logic locking schemes with the topology leakage $\mathcal{L}_{\mathtt{topo}}$, such as our proposed scheme PSYLOCKE, are secure, and Section 6 will demonstrate that these practical attacks indeed require significantly more queries to break PSYLOCKE in practice. Ideally, concrete lower bounds against such real-world adversaries should be derived. However, it is worth noting that, historically, lower bounds on the secret key sizes or communication complexity in cryptographic protocols were first proven against unbounded adversaries and were later refined for polynomial-time adversaries as the field evolved. In this context, we believe our work constitutes the first formal analysis of query complexity in the 17-year history of logic locking. Quantifying the gap between theoretical and practical adversaries remains a challenging open problem, but our analysis provides a solid foundation for advancing this line of research and deepening our understanding of attack resilience.

Why Our Definitions of Attack Resilience Are Not Based on Probability. Most prior security definitions targeting a specific class of attacks evaluate the security level or define the adversary's success condition in terms of probability. One might, therefore, wonder why our definitions of KRA/CRA resilience use the minimum number of oracle queries as the central measure. This choice is closely tied to the discussion in the previous paragraph, namely, the gap between the theoretical power of adversaries and the capabilities of practical adversaries. Moreover, as discussed in Section 4.1, secret keys are not always chosen uniformly at random, and this fact further influences the adversary's success probability. Let us elaborate on this. In general, the use of uniform randomness, such as uniformly-chosen secret keys, is crucial in encryption schemes to ensure confidentiality, as it conceals the underlying probability distribution of the plaintexts. For instance, the one-time pad with non-uniform keys may leak information on probability distributions of the plaintext, thereby increasing the adversary's success probability.[7] This principle motivates our two-tiered approach:

[7] This is an intuitive example, as the perfect secrecy of the one-time pad is not measured by the success probability of adversaries' attacks. One may also consider information-theoretically message authentication codes [Sim85] as another example; if an authentication tag were created with non-uniform secret keys, the probability of success in forgery would be higher.

1. For general security (Definitions 3 and 4), following the traditional approach, we adopt probability-based definitions, which ensure that no information about the original circuit $\mathbb{C}$ is leaked beyond the allowable leakage $\mathcal{L}$. The above concern can be ignored since leaked information stemming from the non-uniformity of secret keys can be taken over by $\mathcal{L}$.

2. For KRA and CRA resilience under $\mathcal{L}$ (Definitions 6 and 7), the non-uniformity of secret keys in $\mathcal{L}$ can directly influence the adversary's success probability. Thus, rather than relying on probability thresholds, our definitions measure resilience by the minimum number of oracle queries required for successful KRA or CRA. We believe this shift provides a practical and meaningful measure of security even in settings with non-uniform randomness.

By focusing on the query complexity in the worst case, our definitions provide a more concrete and implementation-relevant metric that avoids reliance on assumptions about adversarial behavior or distributions of circuits, which may not hold in real-world settings.

5 PSYLOCKE: Provably Secure Logic Locking Construction

We present our provably secure logic locking construction, PSYLOCKE, which can be viewed as an abstraction and formal treatment of previous logic locking schemes based on lookup tables (LUTs), such as LUT-Lock [MZG+18]. While PSYLOCKE may not be entirely novel in that regard, it is important to highlight that PSYLOCKE is *provably secure* under the circuit topology leakage $\mathcal{L}_{\mathtt{topo}}$, which implies provable resilience of PSYLOCKE against key recovery and circuit recovery attacks.

Universal Gates. We define *universal gates* for a building block.

Definition 9 (Universal Gate)**.** *A universal gate* UG *with two binary inputs* $x := (x_1, x_2) \in \{0,1\}^2$ *and four programming bit* $p := (p_1, p_2, p_3, p_4) \in \{0,1\}^4$ *can simulate any (2-input) logic gates* $g : \{0,1\}^2 \to \{0,1\}$*, i.e., there exists* $p \in \{0,1\}^4$ *such that it holds* $\mathsf{UG}(x,p) = g(x)$ *for any input* $x \in \{0,1\}^2$*. For notational convenience, we define an algorithm* Program *that takes any Boolean gate* g *as input and outputs the corresponding programming bit* $p \in \{0,1\}^4$*.*

It is well known that universal gates can be implemented with six AND and three XOR gates in the form of

$$\begin{aligned}\mathsf{UG}(x_1, x_2, p_1, p_2, p_3, p_4) &= (\overline{x_1} \cdot \overline{x_2} \cdot p_1) + (\overline{x_1} \cdot x_2 \cdot p_2) + (x_1 \cdot \overline{x_2} \cdot p_3) + (x_1 \cdot x_2 \cdot p_4) \\ &= (\overline{x_1} \cdot ((\overline{x_2} \cdot p_1) \oplus (x_2 \cdot p_2))) \oplus (x_1 \cdot ((\overline{x_2} \cdot p_3) \oplus (x_2 \cdot p_4))),\end{aligned}$$

where '$\cdot$', '$+$', and '$\oplus$' denote AND, OR, XOR, respectively [KS16, DGS+23].

5.1 Description of PSYLOCKE

Figure 8 gives a description of PSYLOCKE using universal gates.

Theorem 4. PSYLOCKE *given in Figure 8 is* $(\infty, 0, \mathcal{L}_{topo})$-IND-LL *secure if* UG *is universal gates.*

Proof. It is sufficient to show that for any adversary A and any two circuits $\mathbb{C}_0, \mathbb{C}_1$ with $\mathcal{L}_{\mathtt{topo}}(\mathbb{C}_0) = \mathcal{L}_{\mathtt{topo}}(\mathbb{C}_1)$ it holds

$$\Pr\left[\begin{array}{c} b \xleftarrow{\$} \{0,1\} \\ (\mathbb{L}, \mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C}_b) \\ b' \leftarrow \mathsf{A}(\mathsf{st}, \mathbb{L}, \mathcal{L}_{\mathtt{topo}}(\mathbb{C}_b)) \end{array} : b' = b\right] = \frac{1}{2}.$$

Algorithm: $\mathsf{Lock}(1^\lambda, \mathbb{C})$

$\mathbb{L} := \mathbb{C}$ // Fix the topology of $\mathbb{C}$
$\mathsf{k} := \varepsilon$
for $\forall g_i \in \mathbb{C}$ **do**
  $p_i \leftarrow \mathsf{Program}(g_i)$
  Replace g_i in $\mathbb{L}$ with UG
  $\mathsf{k} := (\mathsf{k}, p_i)$
return $(\mathbb{L}, \mathsf{k})$

Figure 8: PSYLOCKE.

$\mathbb{C}_0$ and $\mathbb{C}_1$ satisfy $\mathcal{L}_{\mathsf{topo}}(\mathbb{C}_0) = \mathcal{L}_{\mathsf{topo}}(\mathbb{C}_1)$, i.e., they have the same topology. Therefore, the only differences between $\mathbb{C}_0$ and $\mathbb{C}_1$ are the gates themselves, which are replaced with universal gates. By definition of universal gates, UG realizes any Boolean gate, and hence, it holds $\mathbb{L}_0 = \mathbb{L}_1$ for any $\mathbb{C}_0, \mathbb{C}_1$ with $\mathbb{C}_0 \neq \mathbb{C}_1$. Thus, for any $\mathbb{C}_0, \mathbb{C}_1$ with $\mathcal{L}_{\mathsf{topo}}(\mathbb{C}_0) = \mathcal{L}_{\mathsf{topo}}(\mathbb{C}_1)$, it holds

$$\Pr\left[\begin{array}{c}(\mathbb{L},\mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C}_0) \\ b' \leftarrow \mathsf{A}(\mathsf{st}, \mathbb{L}, \mathcal{L}_{\mathsf{topo}}(\mathbb{C}_0))\end{array} : b' = 0\right] = \Pr\left[\begin{array}{c}(\mathbb{L},\mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C}_1) \\ b' \leftarrow \mathsf{A}(\mathsf{st}, \mathbb{L}, \mathcal{L}_{\mathsf{topo}}(\mathbb{C}_1))\end{array} : b' = 0\right].$$

Hence, we have

$$\begin{aligned}
&\Pr\left[\begin{array}{c} b \xleftarrow{\$} \{0,1\} \\ (\mathbb{L},\mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C}_b) \\ b' \leftarrow \mathsf{A}(\mathsf{st}, \mathbb{L}, \mathcal{L}_{\mathsf{topo}}(\mathbb{C}_b))\end{array} : b' = b\right] \\
&= \frac{1}{2}\Pr\left[\begin{array}{c}(\mathbb{L},\mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C}_0) \\ b' \leftarrow \mathsf{A}(\mathsf{st}, \mathbb{L}, \mathcal{L}_{\mathsf{topo}}(\mathbb{C}_0))\end{array} : b' = 0\right] + \frac{1}{2}\Pr\left[\begin{array}{c}(\mathbb{L},\mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C}_1) \\ b' \leftarrow \mathsf{A}(\mathsf{st}, \mathbb{L}, \mathcal{L}_{\mathsf{topo}}(\mathbb{C}_1))\end{array} : b' = 1\right] \\
&= \frac{1}{2}\left(\Pr\left[\begin{array}{c}(\mathbb{L},\mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C}_0) \\ b' \leftarrow \mathsf{A}(\mathsf{st}, \mathbb{L}, \mathcal{L}_{\mathsf{topo}}(\mathbb{C}_0))\end{array} : b' = 0\right] + \left(1 - \Pr\left[\begin{array}{c}(\mathbb{L},\mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C}_1) \\ b' \leftarrow \mathsf{A}(\mathsf{st}, \mathbb{L}, \mathcal{L}_{\mathsf{topo}}(\mathbb{C}_1))\end{array} : b' = 0\right]\right)\right) \\
&= \frac{1}{2}.
\end{aligned}$$

□

We obtain the following corollary from Theorem 3.

Corollary 1. PSYLOCKE *is* (∞, γ, η)*-CRA-resilient (and therefore* (∞, γ, η)*-KRA-resilient) for any* $\gamma \in (0, 1]$*, where* $\eta := \min\{2\widetilde{\ell_g^*} + 2,\ \gamma \cdot 2^{\ell_i}\}$.

5.2 Efficiency Comparison

We show that PSYLOCKE drastically improves efficiency in terms of the number of gates compared to existing provably secure logic locking schemes, IndLock [MJS+22] and UCLock [BGH+22, MGM+22]. The detailed comparison is given in Table 1.

IndLock incorporates $i\mathcal{O}$ as a core component, which leads to a significant blow-up in circuit size. UCLock, by contrast, is based on universal circuits (UCs), most recent constructions [LYZ+21, DGS+23] incur a circuit size overhead of approximately $3\ell \log \ell$, where ℓ is the size of the original circuit. In comparison, our proposed scheme, PSYLOCKE, achieves substantially better efficiency, requiring only $9\ell_g$ of overhead, where ℓ_g denotes the number of gates in the original circuit. Since $\ell = \ell_i + \ell_g + \ell_o$, and given that $\ell_i \geq 2$, and $\ell_o \geq 1$, it follows that $3\ell \log \ell \geq 3(\ell_g + 3)\log(\ell_g + 3)$.

Table 1: Efficiency comparison in terms of the number of gates among PSYLOCKE and existing provably secure logic locking schemes. We employ ISCAS'85 benchmark circuits† and MCNC benchmark circuits [Yan91], as El Massad et al. [MJS+22] estimated the number of gates with them.

Circuits	c432	c499	c880	i4
Original Size (ℓ)	213	275	469	536
#Gates (ℓ_g)	160	202	383	338
IndLock [MJS+22]	2,197,707	2,197,872	2,198,054	2,198,081
UCLock [BGH+22, MGM+22]	4,943	6,686	12,485	14,579
PSYLOCKE	1,440	1,818	3,447	3,042

† https://pld.ttu.ee/diagnostika/labs/iscasb.htm

Table 2: Comparison in terms of the bit-lengths of secret keys among PSYLOCKE and existing provably secure logic locking schemes for the same benchmarks as Table 1.

Circuits	c432	c499	c880	i4
IndLock [MJS+22]	256	256	256	256
UCLock [BGH+22, MGM+22]	4,062	5,266	10,726	13,272
PSYLOCKE	640	808	1,532	1,352

Therefore, PSYLOCKE is more efficient than UCLock when $\ell_g \geq 5$, which ensures $\log(\ell_g + 3) \geq 3$. In practice, for well-known benchmark circuits, Table 1 demonstrates that PSYLOCKE achieves a 3.5x to 4x smaller overhead compared to UCLock, while still maintaining a reasonable level of KRA/CRA resilience. Note that UCLock also requires significantly larger secret key sizes compared to PSYLOCKE.

We also compare the secret-key sizes required by provably secure logic locking schemes in Table 2. Among them, IndLock is the most efficient, requiring only the secret key for its underlying pseudorandom function, which we instantiate as 256 bits. UCLock inherits the key size from its underlying universal circuit (UC). While the recursive structure of the state-of-the-art UC constructions (e.g., [LYZ+21]) makes exact asymptotic expressions difficult, they demand a large number of programming (secret) bits per gate. In contrast, PSYLOCKE requires only $4 \cdot \ell_g$ bits, resulting in significantly smaller keys. For example, on benchmark i4, PSYLOCKE uses 1,352 bits, compared to 13,272 bits in UCLock, a 9.8× reduction.

6 Evaluation

6.1 Setup

Here, we present our experimental setup to validate the security expectations and the effectiveness of our proposed method. For security assessment, we performed both a functional attack and a structural attack. For the former attack, IcySAT-II [SPJ19a] was used on a 14-core Intel Core-i5 processor running at 3.5 GHz with 64 GB of RAM. For the latter attack, Valkyrie [LPS22] was used on an 8-core Intel Xeon processor running at 3.2 GHz with 128 GB of RAM. We also evaluated the overheads incurred to apply a countermeasure in terms of circuit area, delay, and power consumption. The overhead evaluation was conducted by logic synthesis using Synopsys Design Compiler with a clock frequency constraint of 200 MHz and the Nangate 45nm cell library.

Table 3: Brief summary of the ITC'99 benchmark circuits [CRS00]. In, Out, and GC denote inputs, outputs, and gate counts, respectively.

Name	Functionality	In	Out	GC
b14	Viper processor	277	299	9,767
b15	80386 processor	485	519	8,367
b17	3× b15	1,452	1,512	30,777
b20	1× b14 + 1× modified b14	522	512	19,682
b21	2× b14	522	512	20,027
b22	1× b14 + 2× modified b14	767	757	29,162

We compared our method against the following three SAT-resilient logic locking methods (with a key size of 128 bits) and a provably secure logic locking method, for large ITC'99 benchmarks [CRS00] shown in Table 3. All methods locked the combinational part of the sequential benchmark circuits in our evaluation.

- AntiSAT [XS19]: This method adds two logic functions (point-functions) in the circuit to thwart SAT attacks. The two logic functions – g (AND-tree) and $\bar{g}$ (NAND-tree) – are ANDed to form a single wire, which is merged with the original design.
- Corrupt-and-Correct (CAC) [SML+19]: This methods first corrupts the original design using hard-coded AND-trees and then modifies the functionality using a generic correction unit.
- SARLock [YMR+16]: Similarly to AntiSAT, this method adds point-functions to thwart SAT attacks. Its point-functions include a comparator and a masking circuit connected to the original design.
- UCLock [BGH+22, MGM+22]: As discussed in their papers, FPGA is an initial step to map UC on a practical platform. Therefore, in this evaluation, we generated a minimum FPGA size to configure each benchmark circuit using OpenFPGA [TGA+19]. The generated FPGA has a general 2-dimensional mesh array structure composed of configurable logic blocks and configurable routing switches in between logic blocks. Among the benchmarks in Table 3, b14 needed the smallest FPGA (23×23), while b17 needed the largest (93×93).
- PSYLOCKE: We implemented a 2-input & 1-output configurable logic gate to realize the LUT discussed in the previous section. This gate can express 16 different logic functions by the configuration (or key). Then, we used the minimum number of configurable logic gates to realize each benchmark circuit in the table.

6.2 Security Assessment

SAT Attack Resilience. We first analyzed SAT attack resilience of all compared methods with a time-out setting of 48 hours, consistent with prior studies (e.g., [APM+23]). For AntiSAT, CAC, and SARLock, since only a part of the original circuit can be locked with the logic locking key, the key size has to be carefully set. In our evaluation, the key size of 128 bits was sufficient for all of these methods to resist against the SAT attack under the given time-out setting. UCLock and PSYLOCKE were also resistant against the SAT attack.

Structural Attack Resilience. Next, we discuss the results of the structural attack using Valkyrie [LPS22] as shown in Table 4. Valkyrie's attack mechanism is twofold; it first diagnoses

Table 4: Results against the structural attack using Valkyrie [LPS22]. Numbers show the average attack time over 10 attacks (seconds) which was taken only for diagnosis in PSYLOCKE, but for both diagnosis and recovery in the others. ✓ and ✗ denote that the attack succeeded and failed, respectively. In the table, $*$ indicates that the presented time was only diagnosis time.

Name	AntiSAT (SFLT)		CAC (SFLT)		SARLock (SFLT)		PSYLOCKE (SFLT)		PSYLOCKE (DFLT)	
b14	18	✓	12	✓	17	✓	239*	✗	231*	✗
b15	21	✓	13	✓	21	✓	900*	✗	1,018*	✗
b17	28	✓	31	✓	23	✓	2,582*	✗	2,593*	✗
b20	19	✓	19	✓	19	✓	788*	✗	807*	✗
b21	21	✓	13	✓	19	✓	762*	✗	800*	✗
b22	23	✓	14	✓	20	✓	933*	✗	1,186*	✗

a given locked circuit to find "critical signal(s)" that can give a hint for analyzing the circuit structure, and then it recovers the original circuit based on the obtained critical signal(s). Valkyrie can perform two types of attack techniques called SFLT and DFLT, which recover the original circuit through a single or double critical signal(s), respectively – simply speaking, DFLT is a stronger attack than SFLT. If Valkyrie fails to find critical signal(s), it reports only the diagnosis time and halts without trying the recovery phase. As also have been presented in [LPS22], the table shows that all of the three SAT-resilient logic locking methods were broken by SFLT in a short time. This is because they all contain a critical signal that can capture the structural difference between the original circuit and the added (locked or corrupted-and-corrected) circuit. On the other hand, PSYLOCKE is resistant against both of SFLT and DFLT – Valkyrie struggled to find critical signal(s) for a quite longer time but eventually failed to find them because PSYLOCKE does not contain critical signal(s). These results indicate that PSYLOCKE is robust against the type of structural attacks, including Valkyrie, that recovers the original circuit through wire(s) distinguishing the original and added designs. UCLock was omitted from this evaluation as UCLock does not contain critical signal(s) either, by locking not only logic but also wires.

6.3 Area-Power-Delay (APD) overhead

The APD overhead against the original circuit was obtained using Synopsys DC Compiler. Figures 9 and 10 show circuit area and power results of all compared results. Delay results were omitted since all circuits were synthesized to closely match the target clock frequency, yielding negligible overhead.

As shown in Figure 9, the three SAT-resilient logic locking methods had small area overhead (an average of 5% to 7% overhead) for the large benchmarks used in this evaluation as they only partially lock the original circuit using a 128-bit logic locking key. In contrast, UCLock had impractically larger overhead (an average of 605× and up to 1,285×) by using both configurable logic and configurable routing elements to lock the entire circuit. The overhead linearly grows with the increase of the circuit complexity (i.e., the number of inputs/outputs/logic gates; these increases also largely affect the routing area). Although PSYLOCKE also had larger overhead (an average of 11×[8]) than the three SAT-resilient logic locking methods, its overhead is an order of magnitude lower than UCLock's overhead. Also, interestingly, in contrast to UCLock, the PSYLOCKE's overhead is almost constant by locking only the logic gates. Comparing UCLock and PSYLOCKE clearly shows the critical area overhead incurred by locking the wiring through configurable routing elements.

As shown in Figure 10, the power overhead results are similar to the area overhead results. For UCLock, the power overhead was smaller than the area overhead because of some configurable logic

[8]While the results in Table 1 showed 9× overhead shows a logic gates increase, Figure 9 includes the area overhead for both logic gates and wiring.

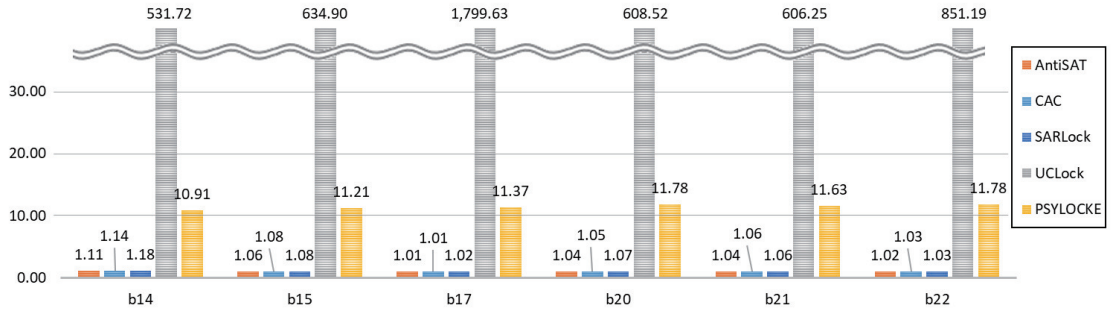


Figure 9: Circuit area overhead. The baseline is the original circuit with no countermeasure applied.

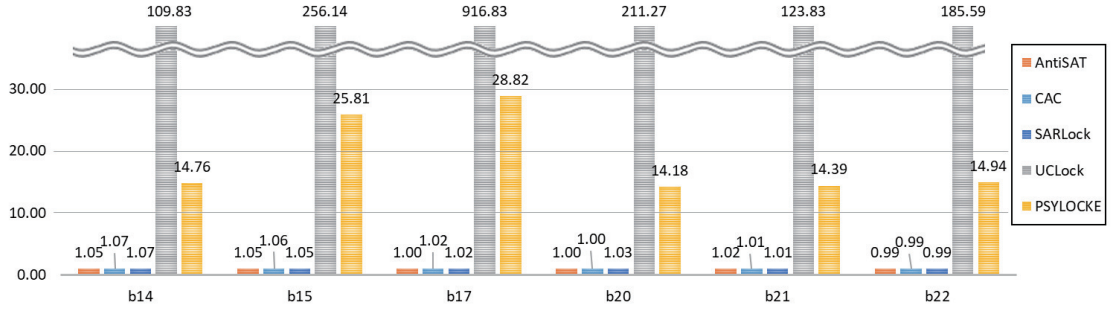


Figure 10: Power overhead. The baseline is the original circuit with no countermeasure applied.

or routing elements that were not fully utilized, resulting in no dynamic power consumption in such elements. However, its overhead is still impractically large, and PSYLOCKE's overhead is an order of magnitude lower than UCLock's overhead.

These APD results indicate the efficiency of PSYLOCKE in that it locks only the logic gates using configurable logic gates.

6.4 Discussions

Based on our security assessment and ADP evaluation shown above, we discuss our insights on the security and implementation efficiency of logic locking methods below.

First of all, from the security assessment results, we can conclude that *structural attacks pose a significant threat to logic locking techniques by analyzing the structural characteristics of a synthesized netlist rather than relying solely on its functional behavior*. Unlike functional attacks such as SAT-based techniques, structural attacks aim to identify and isolate logic locking components by exploiting distinctive patterns in the netlist introduced during synthesis. For example, in CAC, at least three structural properties that can identify critical signals exist; Only one or a few downstream logic gates are driven by critical signals; Critical signals are typically part of short, direct logic paths in proximity to primary outputs; And, the corrupt and restore units usually affect the circuit only under specific protected input patterns. Such irregularities can be exploited through structural heuristics. Furthermore, if the added circuit remains unblended with the original circuit during synthesis, it results in isolated structures that are more easily identifiable and removable by an attacker.

To counter these vulnerabilities, recent approaches have explored new directions for embedding security logic in a structurally indistinguishable manner. For example, the work [CWH25] proposed a logic synthesis optimization method focusing on a CAC-based method so that the corrupt unit can seamlessly merge with the original design. Consequently, the resulting netlist lacks aforementioned

obvious structural signatures, effectively concealing critical signals and rendering Valkyrie-style attacks ineffective. This represents a shift from relying on obfuscation alone to leveraging synthesis-aware transformations for structural security.

In contrast, logic locking schemes such as UCLock and PSYLOCKE take a fundamentally different approach that *inherently defeats structural attacks*. These methods do not inject distinct locking units into specific regions of the circuit. Instead, they perform global, uniform, and unbiased transformations across the entire design. In other words, there is no localized or dedicated perturb unit, and thus no critical signal in the traditional sense. This characteristic fundamentally changes the attack surface: instead of needing to hide or blend critical signal(s), these schemes eliminate its existence altogether. The absence of uniquely structured or functionally sparse regions in the locked design deprives structural attacks of their primary indicators. This structure-neutral transformation in UCLock and PSYLOCKE inherently provides strong resilience against structurally guided reverse engineering attacks.

Here, considering that UCLock and PSYLOCKE are resilient against both functionally and structurally guided attacks, it is natural that they have larger ADP overheads than SAT-resilient but structurally-vulnerable logic locking methods. The big difference between UCLock and PSYLOCKE is, as has been discussed before, whether or not exposure of the circuit topology (wiring) is permitted. The past literature [BGH+22, MGM+22] has claimed that both logic and topology have to be hidden using an FPGA-like platform in order to be resilient against both functional and structural attacks. On the contrary, this paper theoretically and practically demonstrated the resilience of PSYLOCKE against both types of attacks even through exposing the circuit topology. In other words, by permitting to expose the topology and improving implementation efficiency by a magnitude than UCLock, PSYLOCKE achieved a favorable balance between practical performance and security.

7 Conclusion

In this paper, we introduced PSYLOCKE, a new logic locking scheme that achieves provable security under a general class of attacks while maintaining practical efficiency and ASIC compatibility. To this end, we established a new paradigm for formalizing and analyzing the security of logic locking in the presence of explicit allowable leakage $\mathcal{L}$. Our general security definition guarantees that a logic locking scheme leaks no information beyond the leakage $\mathcal{L}$, and our definitions of resilience against specific attacks explicitly quantify the extent to which the leakage $\mathcal{L}$ can be exploited by adversaries in those attacks. We construct PSYLOCKE with universal gates so that it leaks only the circuit topology while preserving functionality and ensuring that no other structural or functional information is revealed. PSYLOCKE requires only $9\ell_g$ overhead, where ℓ_g is the number of gates in the original circuit. This represents a dramatic improvement over UCLock and IndLock, which require circuit size overheads of $3\ell \log \ell$ and several orders of magnitude, respectively.

Furthermore, through quantitative evaluations using widely-used benchmark circuits, we demonstrated that PSYLOCKE reduced the APD overhead by an order of magnitude while achieving the same security level, compared to UCLock. We also provided in-depth insights, especially on the structural characteristics of the locked designs incurred by the different logic locking schemes.

Acknowledgments. This work was supported by JSPS KAKENHI Grant Numbers JP23H00468, JP23H00479, JP23K17455, JP23K21644, JP23KJ0968, JP23K24846, JST CREST JPMJCR23M2, and MEXT Leading Initiative for Excellent Young Researchers.

References

[AFB19] Abdulrahman Alaql, Domenic Forte, and Swarup Bhunia. "Sweep to the Secret: A Constant Propagation Attack on Logic Locking." In: *AsianHost 2019*. IEEE, 2019, pp. 1–6.

[APM+23] Zain Ul Abideen, Tiago Diadami Perez, Mayler Martins, and Samuel Pagliarini. "A Security-Aware and LUT-Based CAD Flow for the Physical Synthesis of hASICs." In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42.10 (2023), pp. 3157–3170.

[BGH+22] Peter Beerel, Marios Georgiou, Ben Hamlin, Alex J. Malozemoff, and Pierluigi Nuzzo. "Towards a Formal Treatment of Logic Locking." In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2022.2 (2022), pp. 92–114.

[BGI+12] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil Vadhan, and Ke Yang. "On the (Im)possibility of Obfuscating Programs." In: *Journal of the ACM* 59.2 (2012).

[BTZ10] Alex Baumgarten, Akhilesh Tyagi, and Joseph Zambreno. "Preventing IC Piracy Using Reconfigurable Logic Barriers." In: *IEEE Des. Test Comput.* 27.1 (2010), pp. 66–75.

[CCJ+20] Hsiao-Yu Chiang, Yung-Chih Chen, De-Xuan Ji, Xiang-Min Yang, Chia-Chun Lin, and Chun-Yao Wang. "LOOPLock: Logic Optimization-Based Cyclic Logic Locking." In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39.10 (2020), pp. 2178–2191.

[CGK+06] Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky. "Searchable Symmetric Encryption: Improved Definitions and Efficient Constructions." In: *ACM SIGSAC Conference on Computer and Communications Security, CCS 2006*. Alexandria, Virginia, USA: ACM, 2006, pp. 79–88.

[CRS00] Fulvio Corno, Matteo Sonza Reorda, and Giovanni Squillero. "RT-level ITC'99 benchmarks and first ATPG results." In: *IEEE Design & Test of Computers* 17.3 (2000), pp. 44–53.

[CS22] Animesh Chhotaray and Thomas Shrimpton. "Hardening Circuit-Design IP Against Reverse-Engineering Attacks." In: *IEEE Symposium on Security and Privacy, S&P 2022*. IEEE, 2022, pp. 1672–1689.

[CSS+19] Giovanni Di Crescenzo, Abhrajit Sengupta, Ozgur Sinanoglu, and Muhammad Yasin. "Logic Locking of Boolean Circuits: Provable Hardware-Based Obfuscation from a Tamper-Proof Memory." In: *SecITC 2019*. Ed. by Emil Simion and Rémi Géraud-Stewart. Vol. 12001. Lecture Notes in Computer Science. Springer, 2019, pp. 172–192.

[CWH25] Hsiang-Chun Cheng, RuiJie Wang, and TingTing Hwang. "Using OFF-set Only for Corrupting Circuit to Resist Structural Attack in CAC Locking." In: *2025 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 2025, pp. 1–6.

[CZH+21] Subhajit Dutta Chowdhury, Gengyu Zhang, Yinghua Hu, and Pierluigi Nuzzo. "Enhancing SAT-Attack Resiliency and Cost-Effectiveness of Reconfigurable-Logic-Based Circuit Obfuscation." In: *IEEE ISCAS 2021*. IEEE, 2021, pp. 1–5.

[DBD+14] Sophie Dupuis, Papa-Sidi Ba, Giorgio Di Natale, Marie-Lise Flottes, and Bruno Rouzeyre. "A novel hardware logic encryption technique for thwarting illegal overproduction and Hardware Trojans." In: *2014 IEEE 20th International On-Line Testing Symposium (IOLTS)*. 2014, pp. 49–54.

[DGS+23] Yann Disser, Daniel Günther, Thomas Schneider, Maximilian Stillger, Arthur Wigandt, and Hossein Yalame. "Breaking the Size Barrier: Universal Circuits Meet Lookup Tables." In: *Advances in Cryptology – ASIACRYPT 2023*. Singapore: Springer, 2023, pp. 3–37.

[GGM86] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. "How to Construct Random Functions." In: *Journal of the ACM* 33.4 (1986), pp. 792–807.

[KAH+19] Hadi Mardani Kamali, Kimia Zamiri Azar, Houman Homayoun, and Avesta Sasan. "Full-Lock: Hard Distributions of SAT instances for Obfuscating Circuits using Fully Configurable Logic and Routing Blocks." In: *DAC 2019*. 2019, pp. 1–6.

[KKN+19] Gaurav Kolhe, Hadi Mardani Kamali, Miklesh Naicker, Tyler David Sheaves, Hamid Mahmoodi, Sai Manoj P. D., Houman Homayoun, Setareh Rafatirad, and Avesta Sasan. "Security and Complexity Analysis of LUT-based Obfuscation: From Blueprint to Reality." In: *ICCAD 2019*. Ed. by David Z. Pan. ACM, 2019, pp. 1–8.

[KPR12] Seny Kamara, Charalampos Papamanthou, and Tom Roeder. "Dynamic Searchable Symmetric Encryption." In: *ACM SIGSAC Conference on Computer and Communications Security, CCS 2012*. Raleigh, North Carolina, USA: ACM, 2012, pp. 965–976.

[KS16] Ágnes Kiss and Thomas Schneider. "Valiant's Universal Circuit is Practical." In: *Advances in Cryptology – EUROCRYPT 2016*. Ed. by Marc Fischlin and Jean-Sébastien Coron. Vol. 9665. Lecture Notes in Computer Science. Springer, 2016, pp. 699–728.

[LMS16] Helger Lipmaa, Payman Mohassel, and Seyed Saeed Sadeghian. "Valiant's Universal Circuit: Improvements, Implementation, and Applications." In: *IACR Cryptol. ePrint Arch.* (2016), p. 17.

[LPS22] Nimisha Limaye, Satwik Patnaik, and Ozgur Sinanoglu. "Valkyrie: Vulnerability Assessment Tool and Attack for Provably-Secure Logic Locking Techniques." In: *IEEE Transactions on Information Forensics and Security* 17 (2022), pp. 744–759.

[LYZ+21] Hanlin Liu, Yu Yu, Shuoyao Zhao, Jiang Zhang, Wenling Liu, and Zhenkai Hu. "Pushing the Limits of Valiant's Universal Circuits: Simpler, Tighter and More Compact." In: *Advances in Cryptology – CRYPTO 2021*. Ed. by Tal Malkin and Chris Peikert. Cham: Springer International Publishing, 2021, pp. 365–394.

[MAS+21] Prashanth Mohan, Oguz Atli, Joseph Sweeney, Onur Kibar, Larry Pileggi, and Ken Mai. "Hardware Redaction via Designer-Directed Fine-Grained eFPGA Insertion." In: *DATE 2021*. 2021, pp. 1186–1191.

[MGM+22] Elisaweta Masserova, Deepali Garg, Ken Mai, Lawrence T. Pileggi, Vipul Goyal, and Bryan Parno. "Logic Locking - Connecting Theory and Practice." In: *IACR Cryptol. ePrint Arch.* (2022), p. 545.

[MGT15] Mohamed El Massad, Siddharth Garg, and Mahesh V. Tripunitara. "Integrated Circuit (IC) Decamouflaging: Reverse Engineering Camouflaged ICs within Minutes." In: *NDSS 2015*. The Internet Society, 2015.

[MJS+22] Mohamed El Massad, Nahid Juma, Jonathan Shahen, Mariana Raykova, Siddharth Garg, and Mahesh Tripunitara. "Locked Circuit Indistinguishability: A Notion of Security for Logic Locking." In: *IEEE CSF 2022*. 2022, pp. 455–470.

[MZG+18] Hadi Mardani Kamali, Kimia Zamiri Azar, Kris Gaj, Houman Homayoun, and Avesta Sasan. "LUT-Lock: A Novel LUT-Based Logic Obfuscation for FPGA-Bitstream and ASIC-Hardware Protection." In: *IEEE ISVLSI 2018*. 2018, pp. 405–410.

[PM15] Stephen M. Plaza and Igor L. Markov. "Solving the Third-Shift Problem in IC Piracy With Test-Aware Logic Locking." In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 34.6 (2015), pp. 961–971.

[RKM08] Jarrod A. Roy, Farinaz Koushanfar, and Igor L. Markov. "EPIC: Ending Piracy of Integrated Circuits." In: *DATE 2008*. 2008, pp. 1069–1074.

[RPS+12] Jeyavijayan Rajendran, Youngok Pino, Ozgur Sinanoglu, and Ramesh Karri. "Security analysis of logic obfuscation." In: *Proceedings of the 49th Annual Design Automation Conference*. New York, NY, USA: Association for Computing Machinery, 2012, pp. 83–89.

[RZZ+15] Jeyavijayan Rajendran, Huan Zhang, Chi Zhang, Garrett S. Rose, Youngok Pino, Ozgur Sinanoglu, and Ramesh Karri. "Fault Analysis-Based Logic Encryption." In: *IEEE Transactions on Computers* 64.2 (2015), pp. 410–424.

[SCM+22] Akashdeep Saha, Urbi Chatterjee, Debdeep Mukhopadhyay, and Rajat Subhra Chakraborty. "DIP Learning on CAS-Lock: Using Distinguishing Input Patterns for Attacking Logic Locking." In: *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 2022, pp. 688–693.

[Sim85] Gustavus J. Simmons. "Authentication Theory/Coding Theory." English. In: *Advances in Cryptology – CRYPTO' 84*. Ed. by GeorgeRobert Blakley and David Chaum. Vol. 196. Springer Berlin Heidelberg, 1985, pp. 411–431.

[SLM+17] Kaveh Shamsi, Meng Li, Travis Meade, Zheng Zhao, David Z. Pan, and Yier Jin. "AppSAT: Approximately deobfuscating integrated circuits." In: *IEEE HOST 2-17*. 2017, pp. 95–100.

[SML+19] Kaveh Shamsi, Travis Meade, Meng Li, David Z. Pan, and Yier Jin. "On the Approximation Resiliency of Logic Locking and IC Camouflaging Schemes." In: *IEEE Transactions on Information Forensics and Security* 14.2 (2019), pp. 347–359.

[SPJ19a] Kaveh Shamsi, David Z. Pan, and Yier Jin. "IcySAT: Improved SAT-based Attacks on Cyclic Locked Circuits." In: *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. 2019, pp. 1–7.

[SPJ19b] Kaveh Shamsi, David Z. Pan, and Yier Jin. "On the Impossibility of Approximation-Resilient Circuit Locking." In: *IEEE HOST 2019*. 2019, pp. 161–170.

[SRM15] Pramod Subramanyan, Sayak Ray, and Sharad Malik. "Evaluating the security of logic encryption algorithms." In: *IEEE HOST 2015*. 2015, pp. 137–143.

[SZ22] Kaveh Shamsi and Guangwei Zhao. "An Oracle-Less Machine-Learning Attack against Lookup-Table-based Logic Locking." In: *GLSVLSI 2022*. Ed. by Ioannis Savidis, Avesta Sasan, Himanshu Thapliyal, and Ronald F. DeMara. ACM, 2022, pp. 133–137.

[TGA+19] Xifan Tang, Edouard Giacomin, Aurélien Alacchi, Baudouin Chauviere, and Pierre-Emmanuel Gaillardon. "OpenFPGA: An Opensource Framework Enabling Rapid Prototyping of Customizable FPGAs." In: *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*. 2019, pp. 367–374.

[Val76] Leslie G. Valiant. "Universal circuits (Preliminary Report)." In: *STOC '76*. Hershey, Pennsylvania, USA: ACM, 1976, pp. 196–203.

[WSM+16] Theodore Winograd, Hassan Salmani, Hamid Mahmoodi, Kris Gaj, and Houman Homayoun. "Hybrid STT-CMOS designs for reverse-engineering prevention." In: *DAC 2016*. ACM, 2016, 88:1–88:6.

[XS19] Yang Xie and Ankur Srivastava. "Anti-SAT: Mitigating SAT Attack on Logic Locking." In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38.2 (2019), pp. 199–207.

[Yan91] Saeyang Yang. "Logic Synthesis and Optimization Benchmarks User Guide: Version 3.0." Microelectronics Center of North Carolina. 1991.

[YMR+16] Muhammad Yasin, Bodhisatwa Mazumdar, Jeyavijayan J V Rajendran, and Ozgur Sinanoglu. "SARLock: SAT attack resistant logic locking." In: *IEEE HOST 2016*. 2016, pp. 236–241.

[YMS+20] Muhammad Yasin, Bodhisatwa Mazumdar, Ozgur Sinanoglu, and Jeyavijayan Rajendran. "Removal Attacks on Logic Locking and Camouflaging Techniques." In: *IEEE Transactions on Emerging Topics in Computing* 8.2 (2020), pp. 517–532.

[YSN+17] Muhammad Yasin, Abhrajit Sengupta, Mohammed Thari Nabeel, Mohammed Ashraf, Jeyavijayan (JV) Rajendran, and Ozgur Sinanoglu. "Provably-Secure Logic Locking: From Theory To Practice." In: *ACM CCS 2017*. ACM, 2017, pp. 1601–1618.

[YTS19] Fangfei Yang, Ming Tang, and Ozgur Sinanoglu. "Stripped Functionality Logic Locking With Hamming Distance-Based Restore Unit (SFLL-hd) – Unlocked." In: *IEEE Transactions on Information Forensics and Security* 14.10 (2019), pp. 2778–2786.

[ZSL+18] Monir Zaman, Abhrajit Sengupta, Danqing Liu, Ozgur Sinanoglu, Yiorgos Makris, and Jeyavijayan J V Rajendran. "Towards provably-secure performance locking." In: *DATE 2018*. 2018, pp. 1592–1597.

A Previous Definitions and Their Limitations

Removal Attack Resilience. Yasin et al. [YSN+17] formalized resilience against removal attacks, which are a type of structural attack.

Definition 10 (η-Removal Attack Resilience [YSN+17])**.** *Let* Lock *be a logic locking scheme and* $\mathsf{T} : \mathbb{L} \to \mathbb{C}^*$ *be a transformation for the removal attack, where* $\mathbb{L} \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C})$ *for a circuit* $\mathbb{C}$*.* Lock *is called* η*-resilient against removal attacks, where* η *denotes the cardinality of the set of* protected input patterns $\mathcal{P}$*, where it holds* $\mathbb{C}^*(x) \neq \mathbb{C}(x)$ *for any* $x \in \mathcal{P}$*.*

The above definition is, unfortunately, ill-defined since a "transformation for the removal attack" is not formally defined. If any transformation is allowed, there must exist a transformation T^* that output the original circuit $\mathbb{C}$.

Security against Circuit Recovery Attacks. El Massad et al. [MJS+22] defined security against circuit recovery attacks in a similar way to the security definition against key recovery attacks (Definition 5).

Definition 11 (Security against Circuit Recovery Attacks [MJS+22])**.** *A logic locking scheme* Lock *is secure against circuit recovery attacks if there exists a negligible function negl such that for any polynomial-time adversary* A*, a polynomial-time simulator* S *exists such that for every circuit* $\mathbb{C}$*, every sufficiently large* $\lambda \in \mathbb{N}$*, and every polynomial-time algorithm* V *that outputs* 1 *only if it holds*

$$\left| \Pr\left[\begin{matrix} \mathsf{k} \xleftarrow{\$} \{0,1\}^\lambda \\ \mathbb{C}_\mathsf{A} \leftarrow \mathsf{A}^{\mathbb{C}}(\mathsf{Lock}'(\mathsf{k}, \mathbb{C})) \end{matrix} : \mathsf{V}(\mathbb{C}_\mathsf{A}, \mathbb{C}) \to 1 \right] - \Pr\left[\mathbb{C}_\mathsf{S} \leftarrow \mathsf{S}^{\mathbb{C}}(\mathcal{L}_{io}(\mathbb{C}), 1^\lambda) : \mathsf{V}(\mathbb{C}_\mathsf{S}, \mathbb{C}) \to 1 \right] \right| \leq \mathsf{negl}(\lambda).$$

Although the above definition is well-defined and provides a strong security guarantee, there are also several gaps, as in Definition 5.

Functional Secrecy. Beerel et al. [BGH+22] introduced a notion of functional secrecy by refining previous ill-defined security notions [CSS+19, SPJ19b].

Definition 12 (Functional Secrecy [BGH+22])**.** *A logic locking scheme* Lock *is* $(t, q, \varepsilon, \sigma, \tau, \mathcal{L})$*-*FS*-secure if for any* t*-time* q*-query adversary* A*, there exists a* $\mathsf{poly}(t)$*-time* $\mathsf{poly}(q, t)$*-query simulater* S *such that for any circuit* $\mathbb{C}$ *and auxiliary information* aux*, it holds that*

$$\Pr\left[\begin{matrix} (\mathbb{L}, \mathsf{k}) \leftarrow \mathsf{Lock}(1^\lambda, \mathbb{C}) \\ \mathbb{C}_\mathsf{A} \leftarrow \mathsf{A}^{\mathbb{C}}(\mathbb{L}, \mathcal{L}(\mathbb{C}), \mathsf{aux}) \end{matrix} : \mathbb{C}_\mathsf{A} \equiv_\varepsilon \mathbb{C} \right] \geq \sigma$$
$$\implies \Pr\left[\mathbb{C}_\mathsf{S} \leftarrow \mathsf{S}^{\mathbb{C}}(\mathcal{L}(\mathbb{C}), 1^\lambda) : \mathbb{C}_\mathsf{S} \equiv_\varepsilon \mathbb{C} \right] \geq \tau,$$

where '$\mathbb{C}' \equiv_\varepsilon \mathbb{C}$*' indicates that* $\mathbb{C}'$ *agrees with* $\mathbb{C}$ *on at least a fraction* ε *of inputs.*

The above definition is well-defined in the sense that it avoids trivial attacks that would otherwise render the definition unsatisfiable. However, the functional secrecy notion also excludes the consideration of allowable leakage $\mathcal{L}$, and consequently fails to quantify or clarify the extent to which such leakage may aid in circuit recovery attacks.